2020 MathWorks
中国汽车年会

从认证方和工具商角度解读A-SPICE实例

杨环宇
上海先起
樊朝祥
MathWorks中国区应用工程师



MathWorks®

Introduction

杨环宇 (Thomas Yang)

- 上海先起公司首席过程及软件顾问
- 国内最早的Automotive SPICE® 从业人员（2009年）
- 国内第三方中，中国大陆最早获取Automotive SPICE® Principal Assessor资质
- 国内唯一同时具备ASPICE最高级评估师资质及CMMI主任评估师资质的专家
- 国内较早的汽车功能安全从业人员（2012年）
- 软件工程硕士，22年从业经验（车载E/E项目开发管理、过程咨询评估）
- 丰富的CMMI-Dev, Automotive SPICE®, ISO26262项目经验

资质

- intacs™ Certified Automotive SPICE® Principal Assessor
- CMMI Institute Certified CMMI Leader Appraiser
- Functional Safety Professional
- Project Management Professional (PMP)

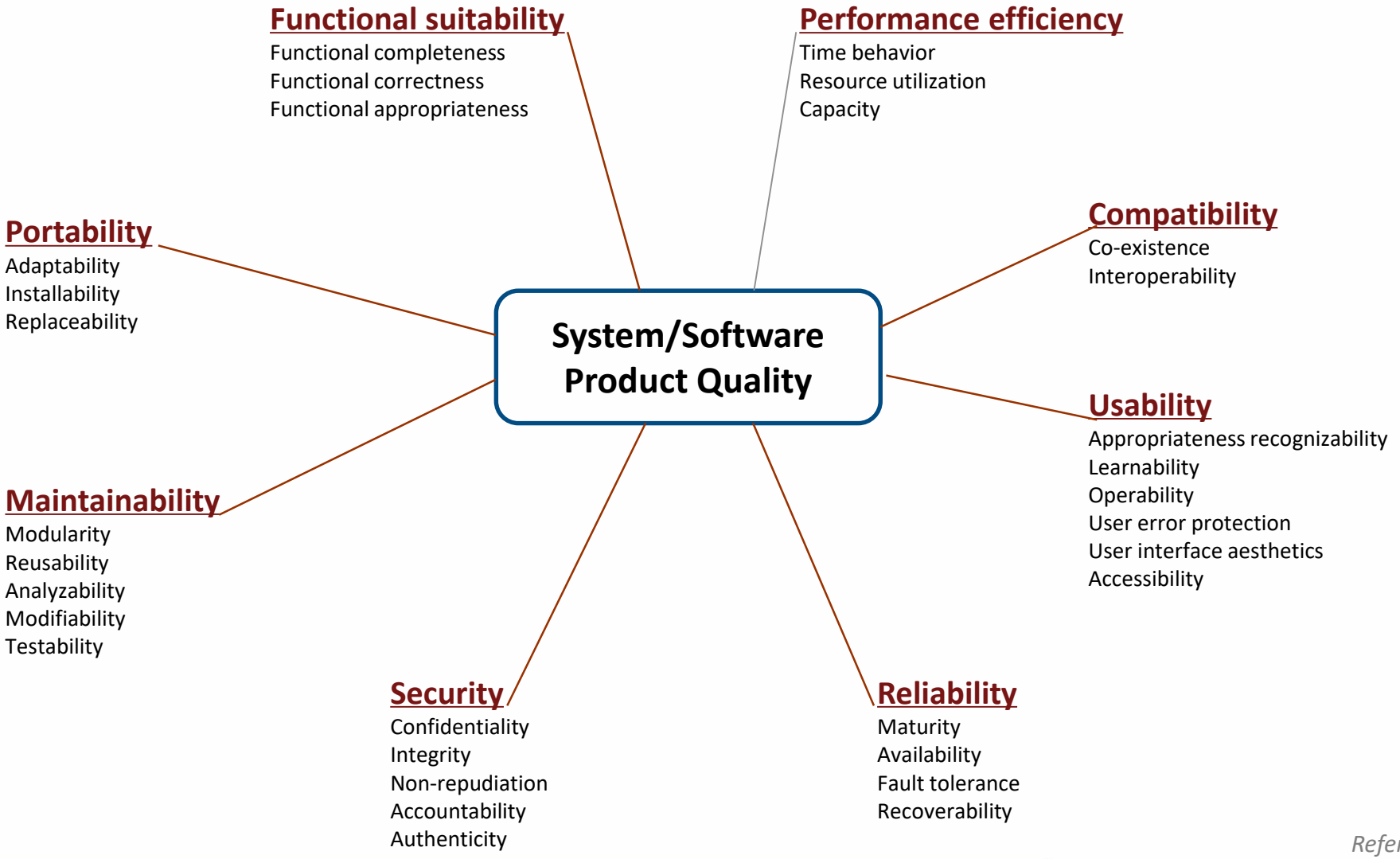
樊朝祥

- MathWorks中国应用工程师
- 10年嵌入式系统软件开发经验
- 主要负责基于模型的设计，测试验证，代码生成相关工作
- 毕业于重庆理工大学，专业方向为软件工程。
- 曾就职于Valeo，从事汽车电子嵌入式系统软件开发工作，在嵌入式系统软件开发，基于模型的设计，软件架构，软件项目管理领域有多年工作经验。

主题

- 背景介绍
- 采用MBD方法，满足ASPICE要求
  - MBD开发概述
  - 详细举例：SWE.3 软件详细设计与单元实现
  - 详细举例：SWE.4 软件单元测试

System/Software Product Quality



Refer from ISO/IEC 25010

Error correction costs today

Typical fault correction during:

concept phase	1	kEuro
A sample	3,5	kEuro
B sample	4	kEuro
C sample	6	kEuro
PV series	65	kEuro
0 series	80	kEuro
series	90	kEuro

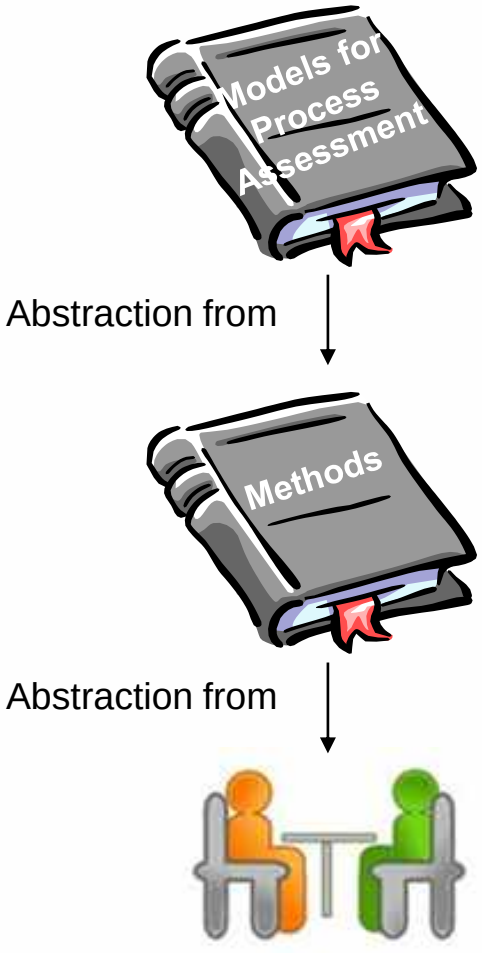
Therefore: resolve defects as early in the process as possible!

Source: HIS (Audi, BMW, Daimler, Porsche and Volkswagen), not considering vehicle modifications like flashing, commissioning etc.

Consensus: the better the processes...

- … the earlier defects are detected
- … the less systematic faults remain in the product
- … the more accurate are the plans & estimates
- … the more predictable is the organization’s performance
- … the more reusable are assets and knowledge/experience
- … the lower is the cost

Levels of abstraction of the term “process”



The “**WHAT**“ (the goals):

(What is to be done, and why, and what are the technical dependencies)

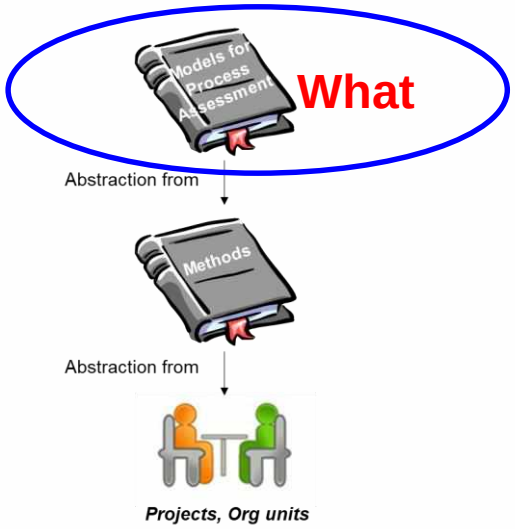
The “**HOW**“ (the way to the goals):

(Lifecycle models, tools, templates, methods, metrics, best practice, guidance, procedures, roles & skills, tailoring guidance, “interweaving“ all this to form workflows)

The “**DOING**“:

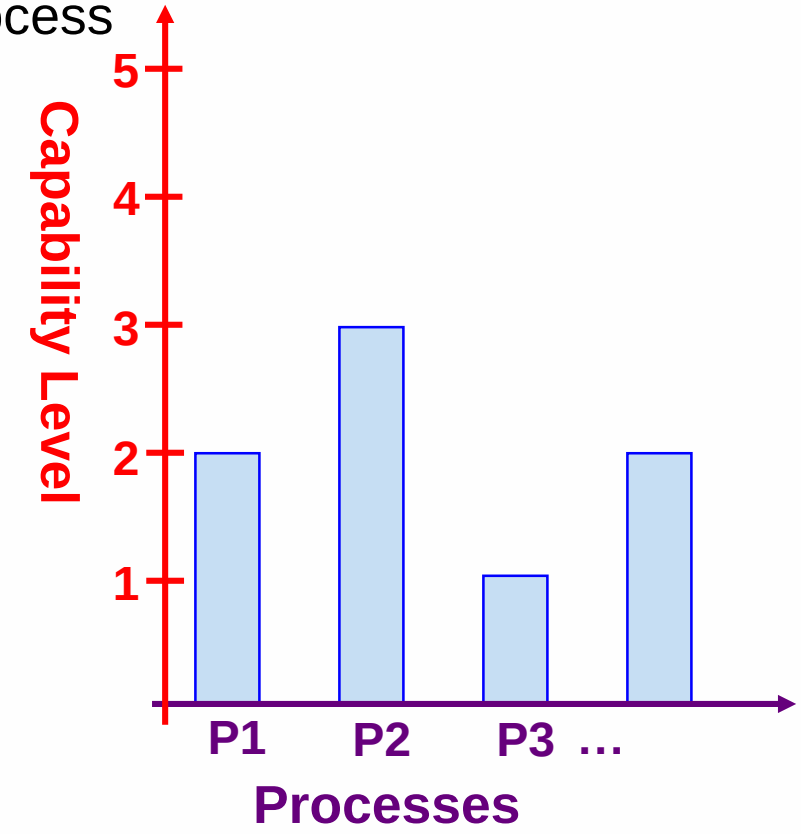
(Tailoring, set-up, and project performance according to the tailored method)

Process in “What” Level - Automotive SPICE®

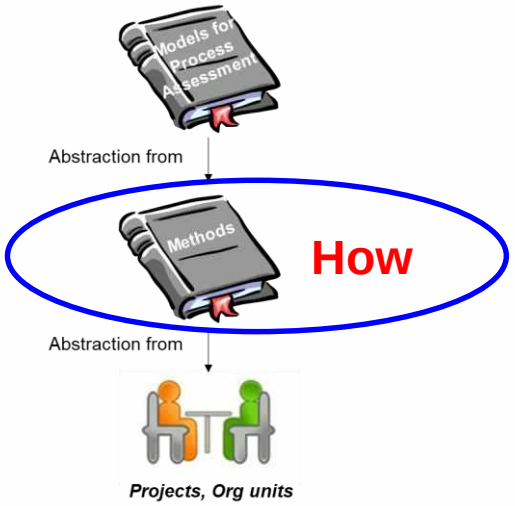


Two dimensional model for Process & Process Capability

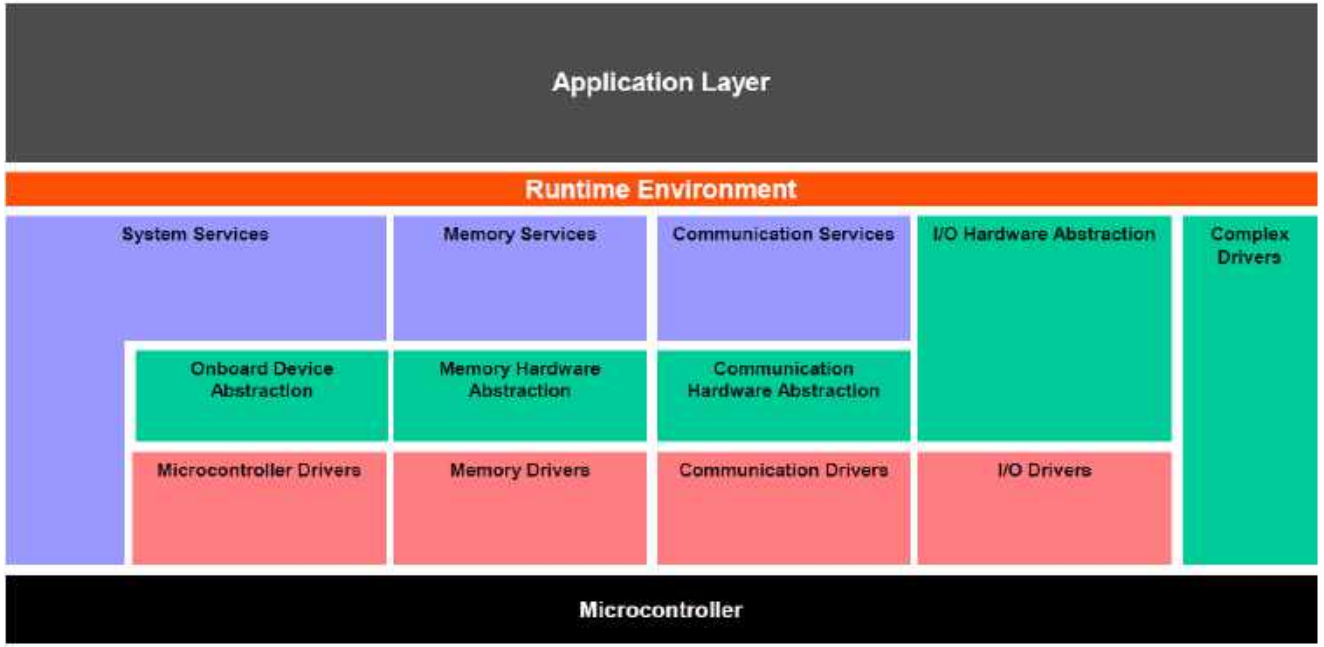
- Process dimension
  - Process Categories
  - Processes (P1, …, Pn)
- Capability dimension
  - Capability Levels (CL 1 , …, CL 5)
  - Process Attributes



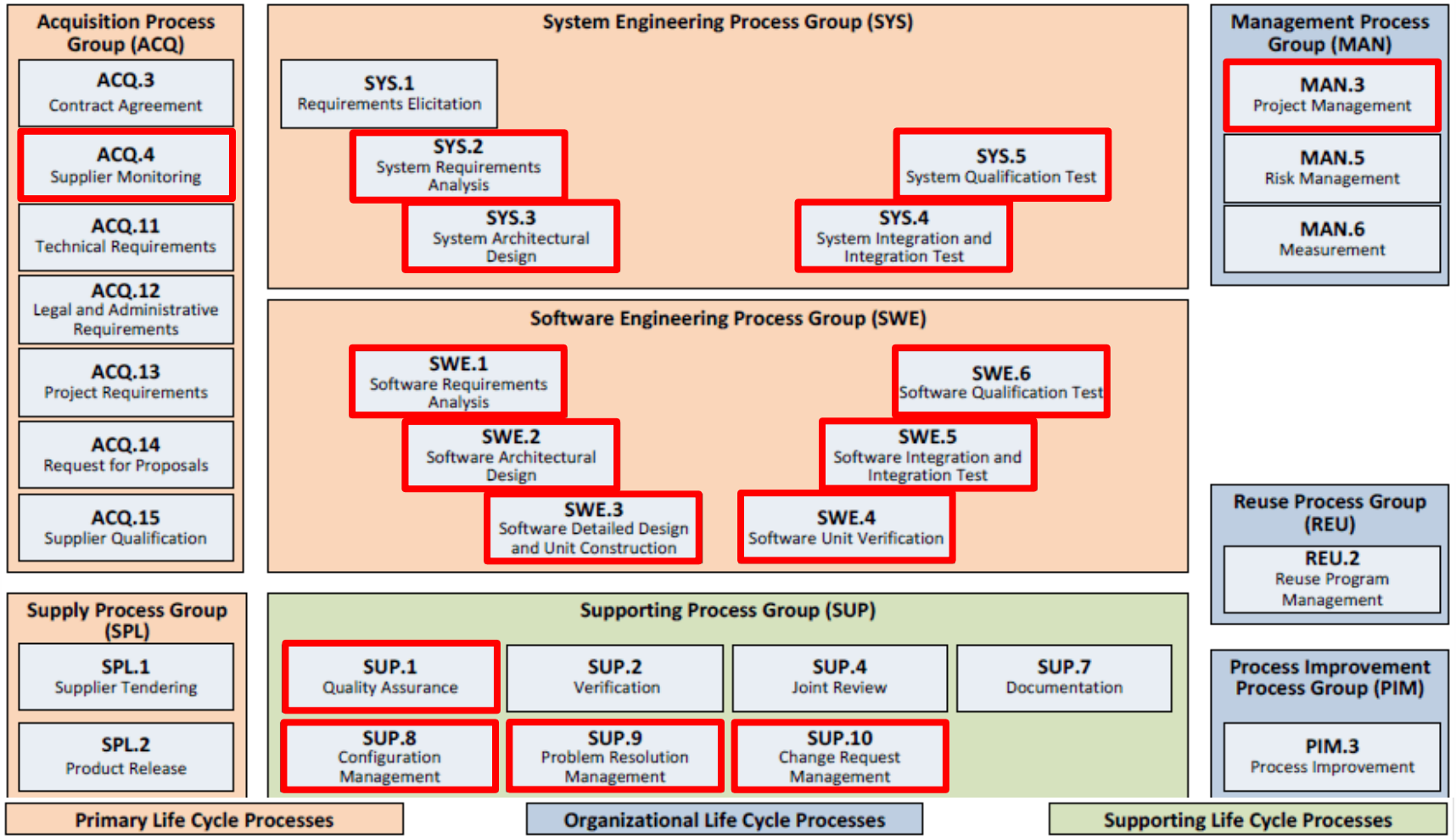
Process in “How” Level - methodology



标准化，专业化，工具化：
- AUTOSAR架构
- 基于模型开发(MBD)



Automotive SPICE® PAM and 'VDA scope'



Red frame = processes of VDA scope

主题

- 背景介绍
- 采用MBD方法，满足ASPICE要求
  - MBD开发概述
  - 详细举例：SWE.3 软件详细设计与单元实现
  - 详细举例：SWE.4 软件单元测试

MBD可用于满足A-SPICE要求



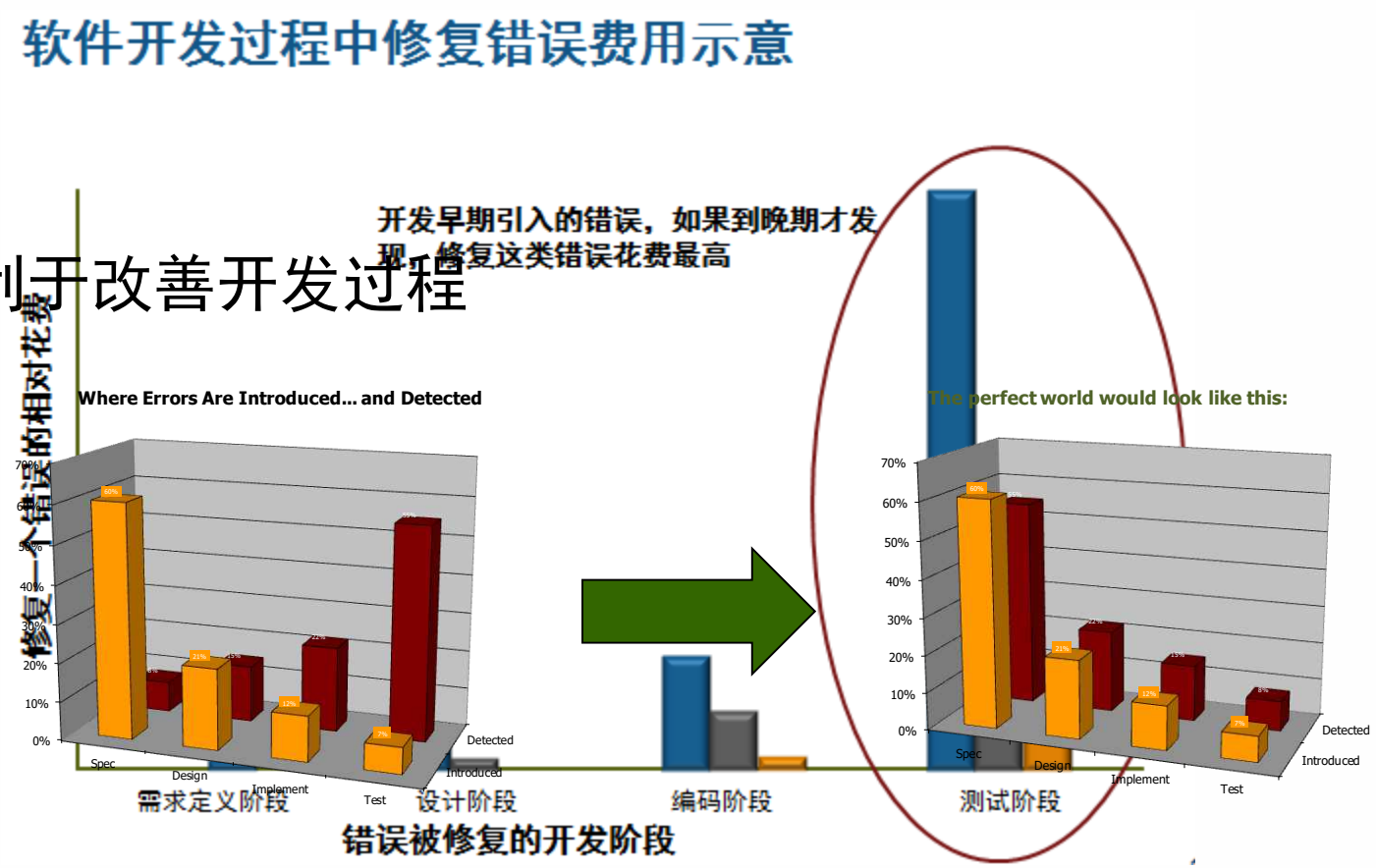
2.2 Application in specific environments

2.2.1 Model based development

The approach of model-based development can be used for different purposes within the system and software development e.g. models can support the requirements elicitation process or support the development of complex algorithms.

早期验证

- 早期引入的错误晚期发现增加修复成本
- 早期验证有利于改善开发过程



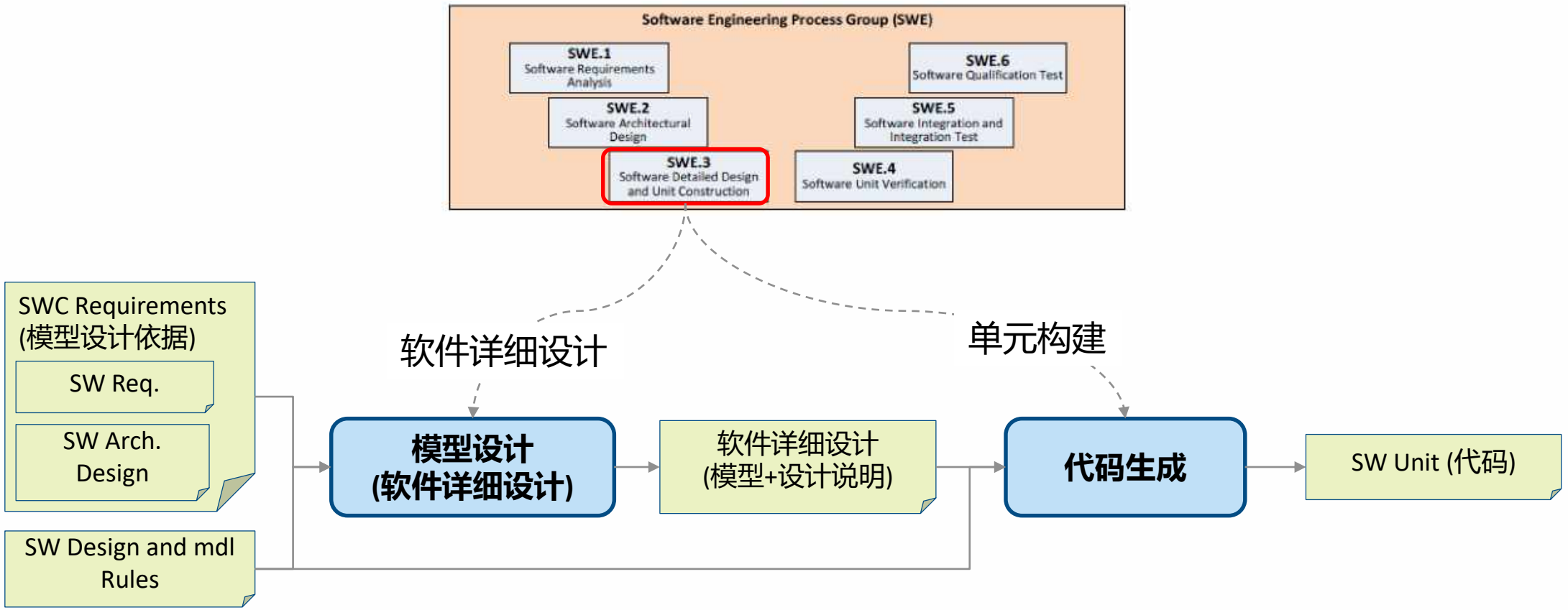
基于模型设计的优势

图形化设计	早期验证	代码自动生成	文档自动化
• 简洁、明确 • 便于交流 • 便于维护	• 及早纠错 • 改善开发过程	• 开发效率 • 代码品质	• 提高效率 • 便于交流 • 改善开发过程

主题

- 背景介绍
- 采用MBD方法，满足ASPICE要求
  - MBD开发概述
  - 详细举例：SWE.3 软件详细设计与单元实现
  - 详细举例：SWE.4 软件单元测试

SWE.3 软件详细设计与单元构建概述(1)



SWE.3 软件详细设计与单元构建概述(2)

- 为每个SWC开发详细设计，设计模型
  - 使用Simulink, Stateflow等
  - 考虑该SWC需要满足的功能性需求和非功能性需求(SWC Req., Design Rules)
  - 通过simulation，评价设计/算法的正确性
  - 通过Model-Advisor Checks，确保满足automotive行业相关准则（如：MAAB, MISRA, ISO26262等)

- 记录相关的设计理由
  - 通过Text形式，在模型上记录设计理由(思路)
  - 建立模型block与之相关的SWC Req.(设计依据)之间的追溯性

SWE.3 基本实践与输出

SWE.3 – 基本实践(BP)

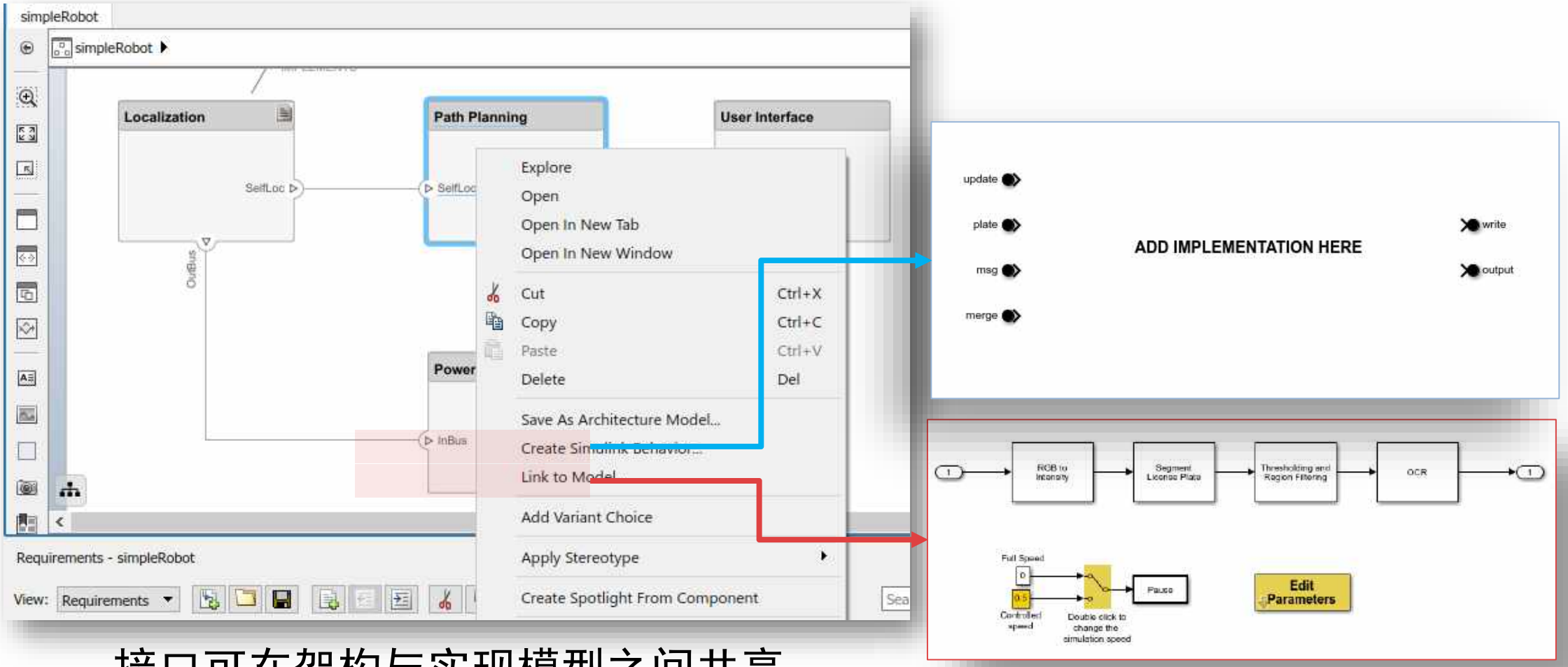
- SWE.3.BP1: 开发软件详细设计
- SWE.3.BP2: 定义软件单元接口
- SWE.3.BP3: 描述动态行为
- SWE.3.BP4: 评估软件详细设计
- SWE.3.BP5: 建立双向追溯性
- SWE.3.BP6: 确保一致性
- SWE.3.BP7: 沟通达成一致的软件详细设计
- SWE.3.BP8: 构建软件单元

实施SWE.3过程的结果如下

- 开发了描述软件单元的详细设计
- 定义了各软件单元的接口
- 定义了软件单元的动态行为
- 建立了双向追溯性和一致性：
  - 软件需求与软件单元之间
  - 软件架构设计与软件详细设计之间
  - 软件详细设计与软件单元之间
- 构建了软件详细设计所定义的软件单元

Model-Based Design and Automotive SPICE

SWE.3.BP1: 开发软件详细设计 – 与架构追溯

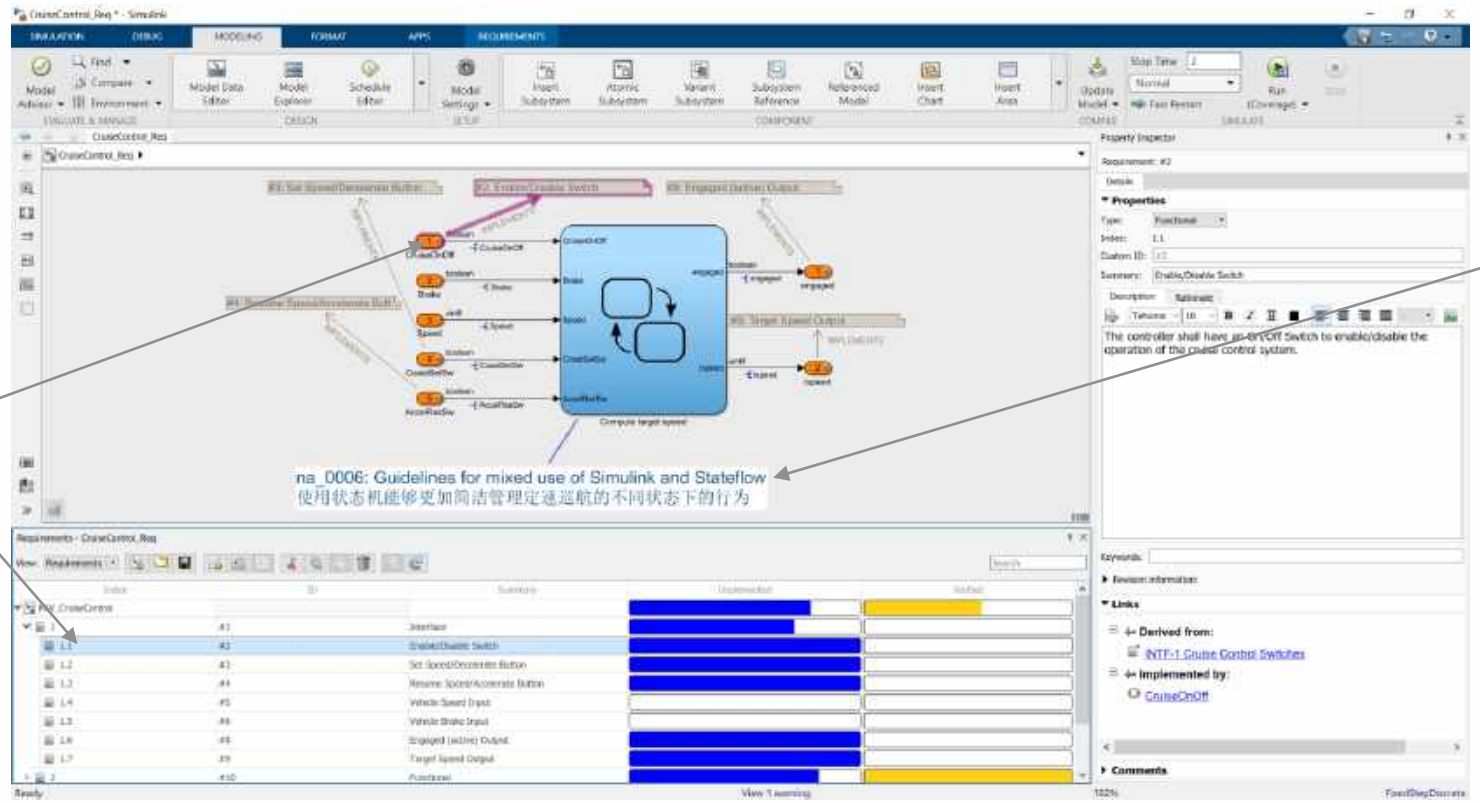


接口可在架构与实现模型之间共享

Model-Based Design and Automotive SPICE

SWE.3.BP1: 开发软件详细设计 – 需求实现与追溯

双向追溯
需求可以从
外部环境
Doors,
Excel,
Word
导入

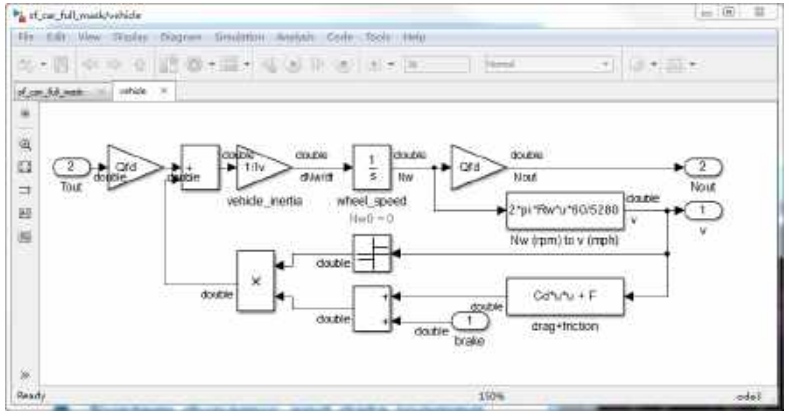


非功能性需求，
设计选择
可以以备注的形式
添加到模型

Model-Based Design and Automotive SPICE

SWE.3.BP1: 开发软件详细设计 – 功能分解

Simulink



MATLAB

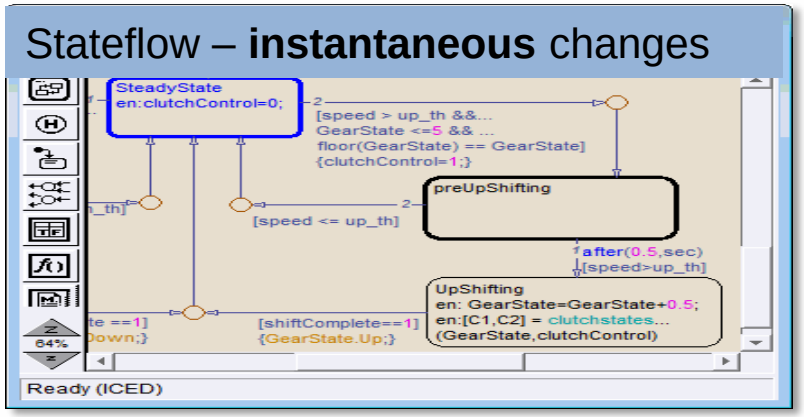
```
function [mean,stdev] = stats(vals)
% #codegen

% calculates a statistical mean and a standard
% deviation for the values in vals.

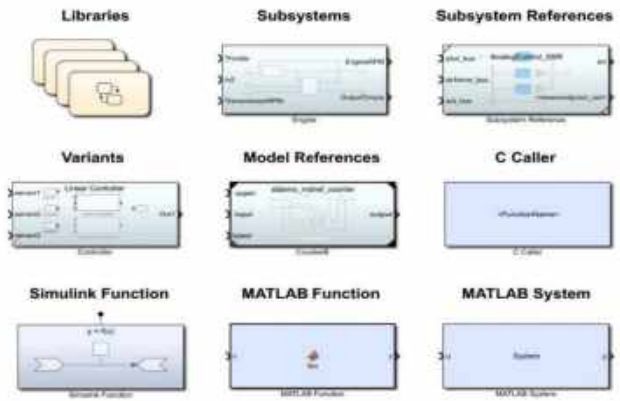
len = length(vals);
mean = avg(vals,len);
stdev = sqrt(sum(((vals-avg(vals,len)).^2))/len);
coder.extrinsic('plot');
plot(vals,'--+');

function mean = avg(array,size)
mean = sum(array)/size;
```

Stateflow



功能分解



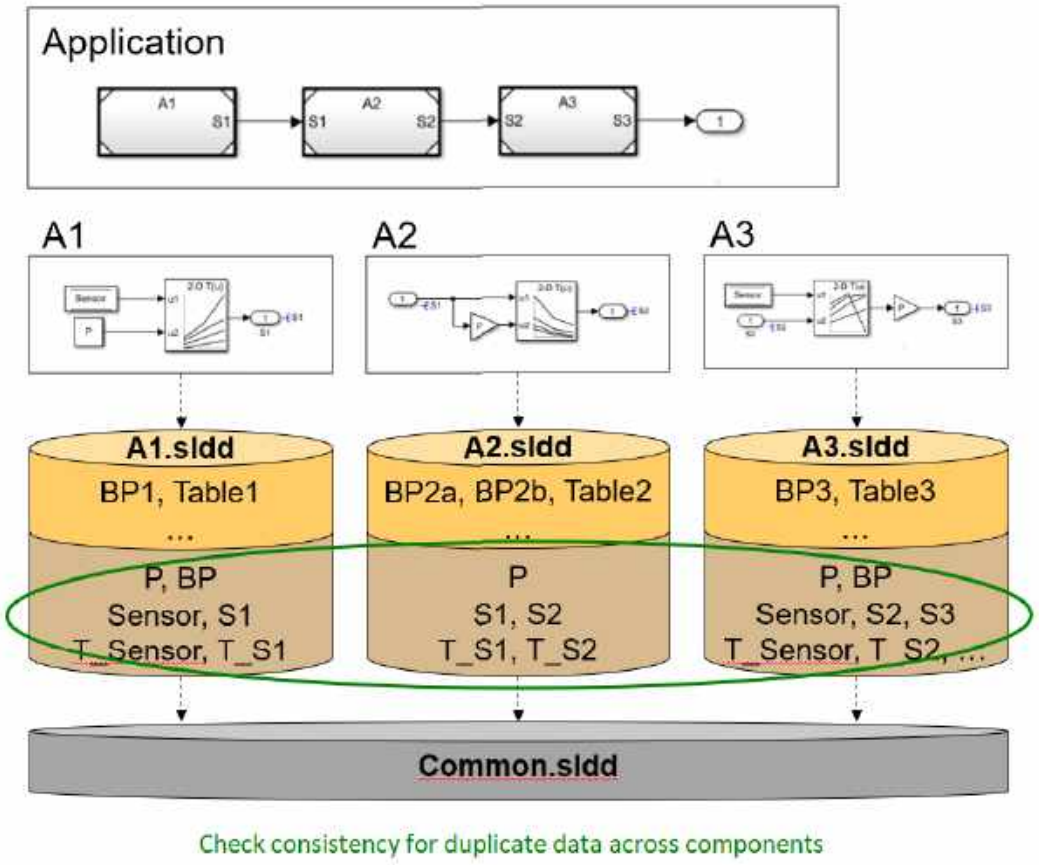
Model-Based Design and Automotive SPICE

SWE.3.BP2: 定义软件单元接口 – 数据字典

- 通过数据字典管理单元接口，标定参数以及观测量

- 数据字典引用：相同接口在多个模型之间维护公用的data dictionary，通过数据字典引用的方式应用到模型，确保接口在不同模型间的一致性
  - R2019a 支持同一接口在多个数据字典重复定义，模型编译期间检查接口的一致性，最终生成代码只有一份定义，解耦数据管理依赖.

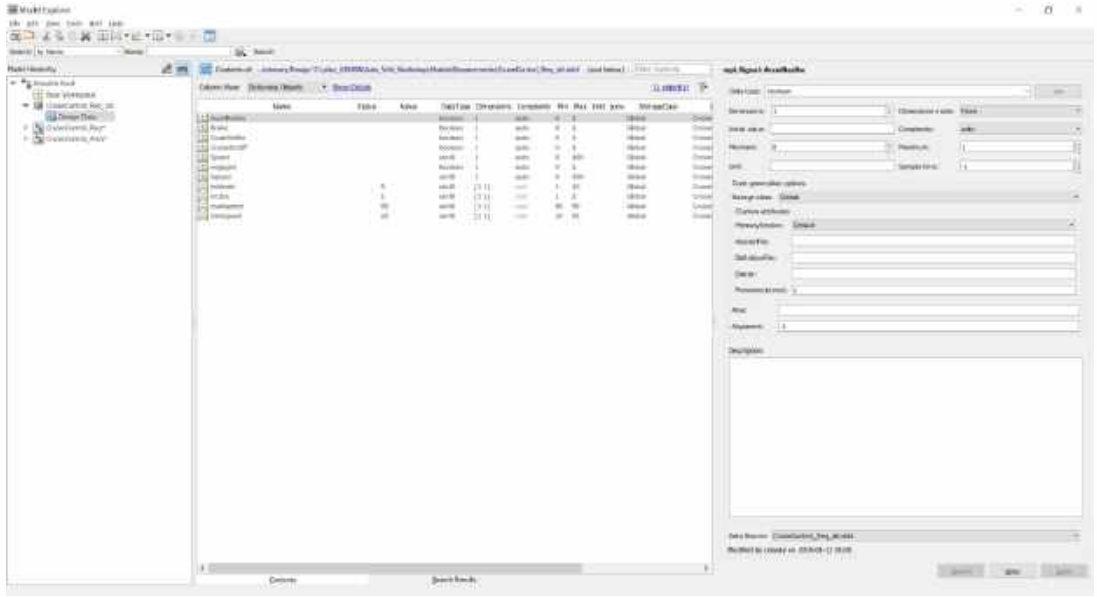
- 模型生成的SDD文档包括data dictionary定义的接口信息，方便阅读和确认接口信息。



Model-Based Design and Automotive SPICE

SWE.3.BP2: 定义软件单元接口 – 接口示例

数据字典



SDD报告接口

3.1 Design Variable Summary

Table 3.1 Design Variables

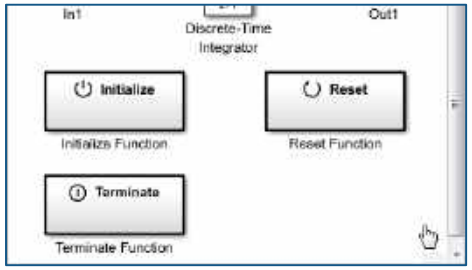
Variable Name	Parent Blocks	Size	Bytes	Class	Value
AccelResSw	AccelResSw	1x1	8	mpt.Signal	< mpt.Signal>
Brake	Brake	1x1	8	mpt.Signal	< mpt.Signal>
CoastSetSw	CoastSetSw	1x1	8	mpt.Signal	< mpt.Signal>
CruiseOnOff	CruiseOnOff	1x1	8	mpt.Signal	< mpt.Signal>
Speed	Speed	1x1	8	mpt.Signal	< mpt.Signal>
engaged	Compute target speed	1x1	8	mpt.Signal	< mpt.Signal>
holdrate	Compute target speed	1x1	1	uint8	5
incdec	Compute target speed	1x1	1	uint8	1
maxtspeed	Compute target speed	1x1	1	uint8	90
mintspeed	Compute target speed	1x1	1	uint8	20
tspeed	Compute target speed	1x1	8	mpt.Signal	< mpt.Signal>

Model-Based Design and Automotive SPICE

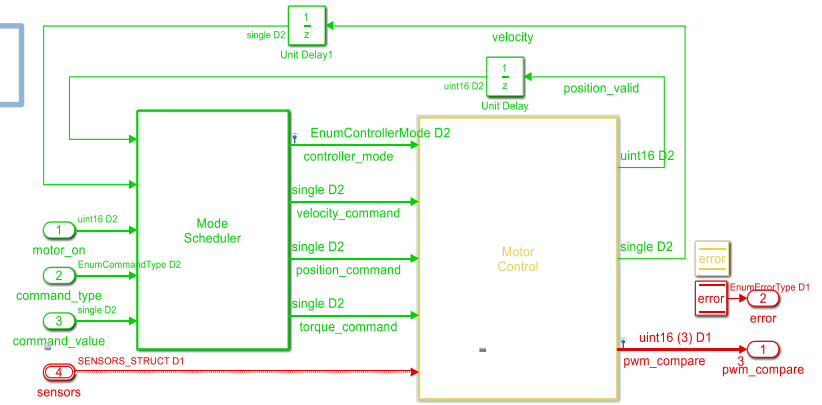
SWE.3.BP3: 描述动态行为

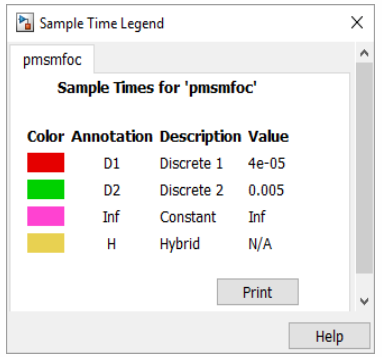
1. 应用不同操作模式函数Initialize, Reset, Terminate
2. 给子系统指定不同的采样周期
3. 通过Stateflow输出显性调度功能模块
4. 使用Schedule Editor 可视化调度

1

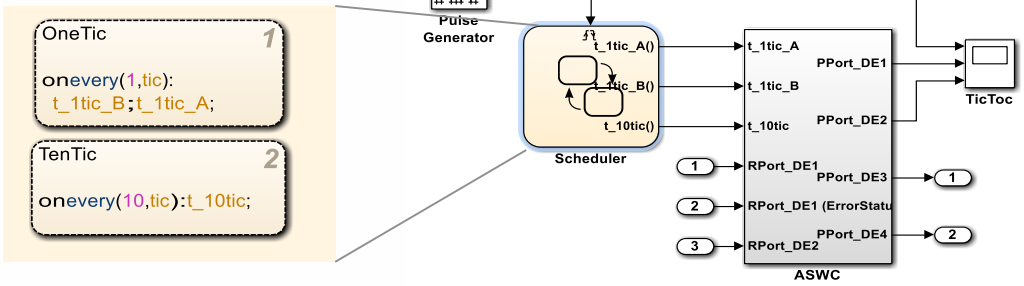


2

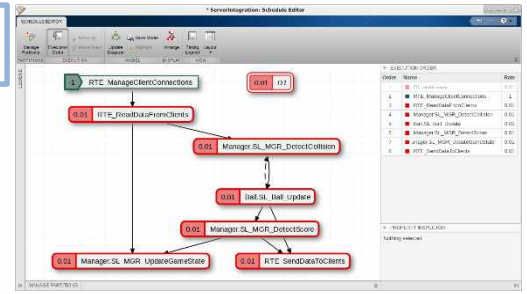




3



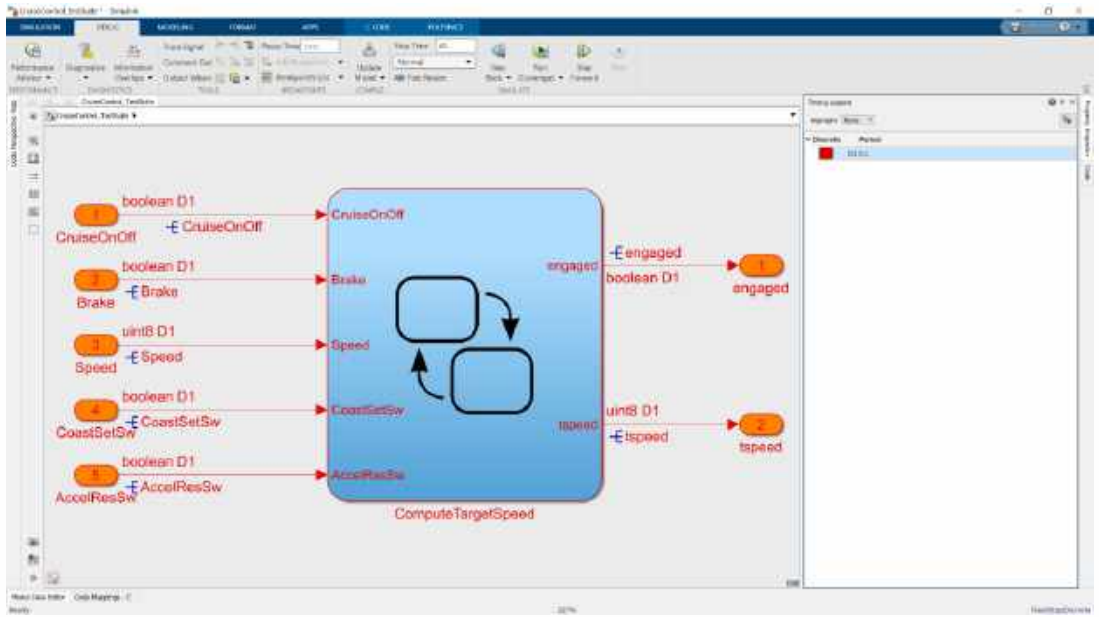
4



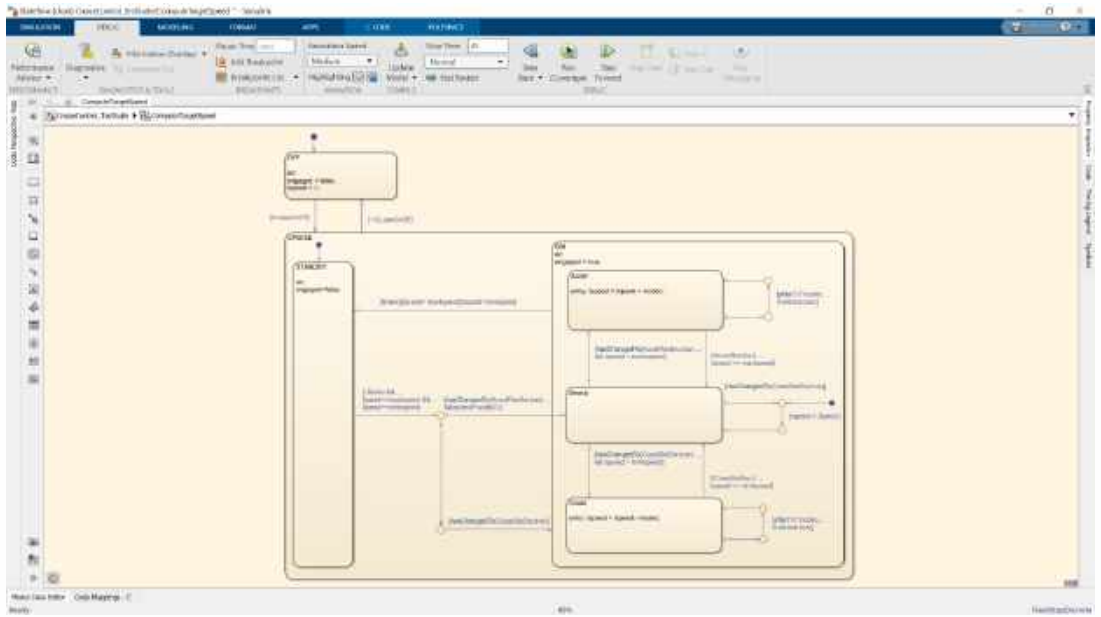
Model-Based Design and Automotive SPICE

SWE.3.BP3: 描述动态行为 - 示例

调度周期



内部交互

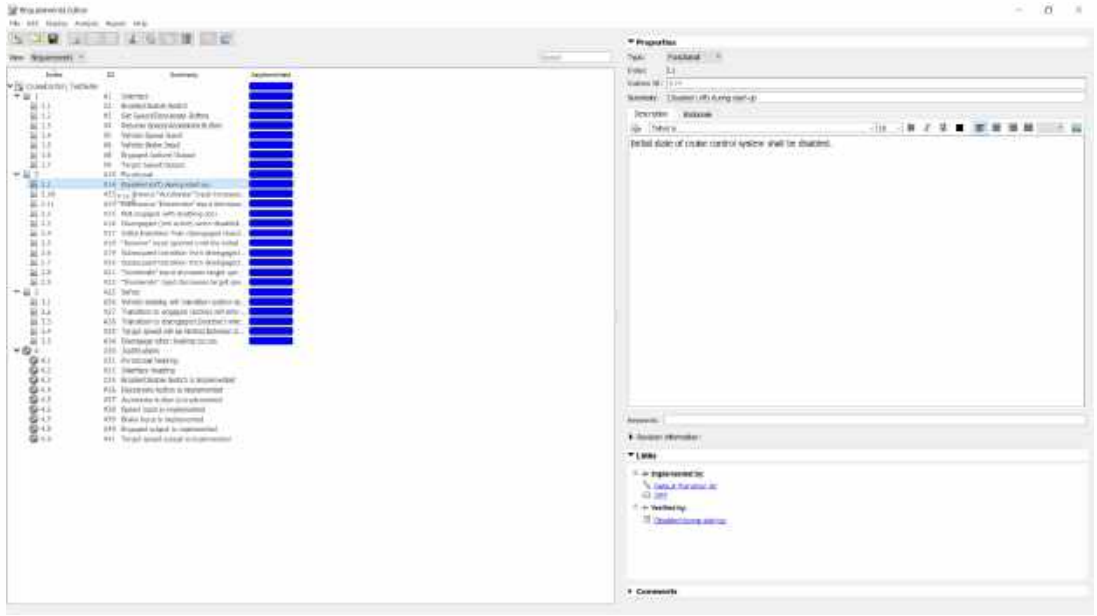


Model-Based Design and Automotive SPICE

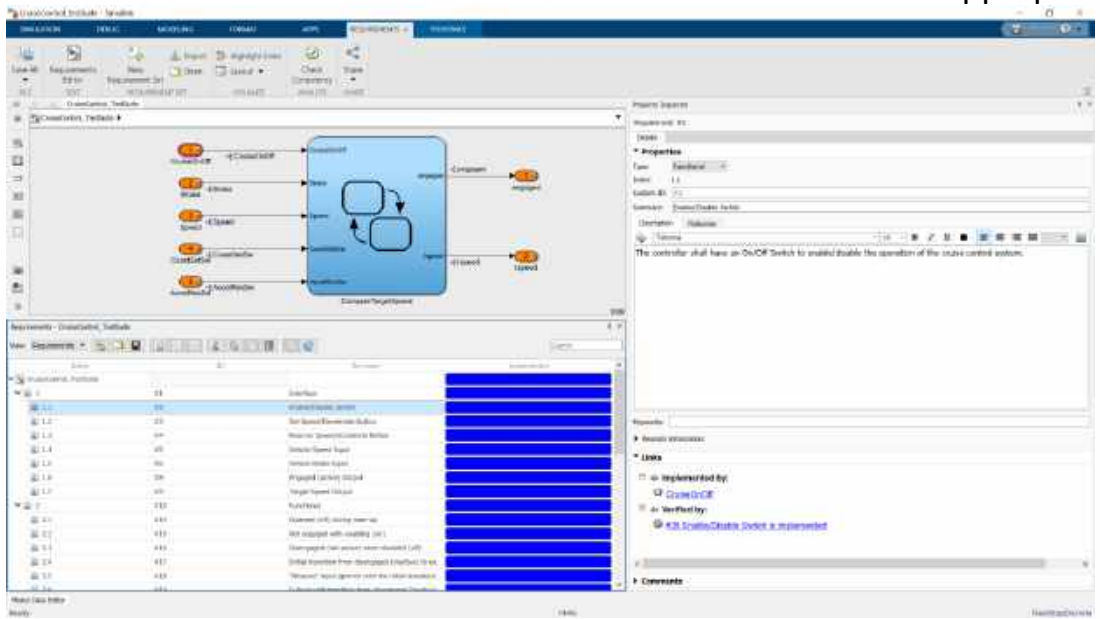
SWE.3.BP4: 评估软件详细设计 – 双向追溯

- 评审模型，并确认是否符合与之追溯的SWC Req. (设计依据)

Functional suitability
Functional completeness
Functional correctness
Functional appropriateness



需求编辑器视图

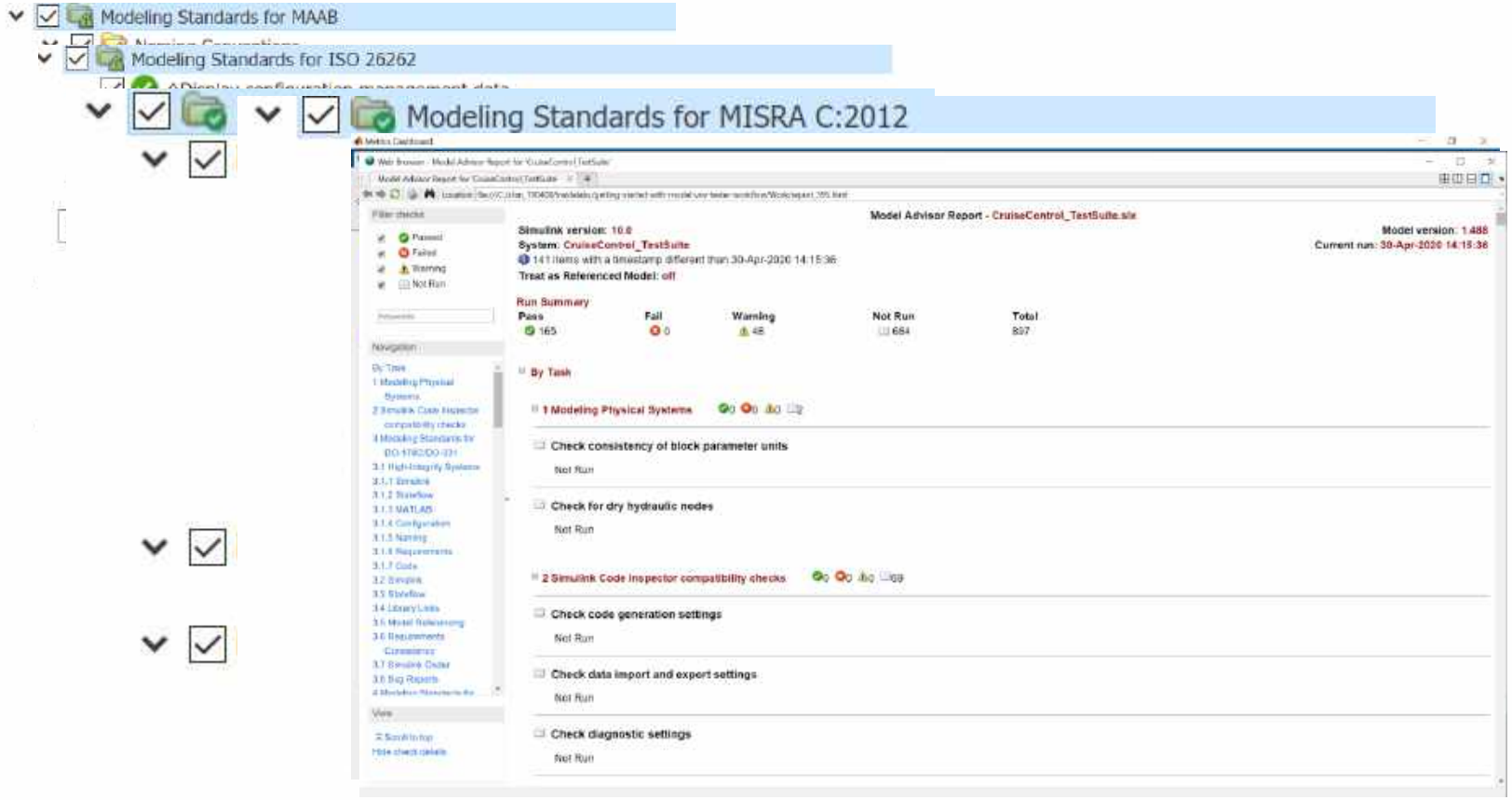


模型需求透视图

Model-Based Design and Automotive SPICE

SWE.3.BP4: 评估软件详细设计 – 建模规范

- 通过Model-metrics，评价模型的复杂度、规模等
- 评价模型与相关行业标准的符合性（如：ISO26262, MISRA, MAAB等）



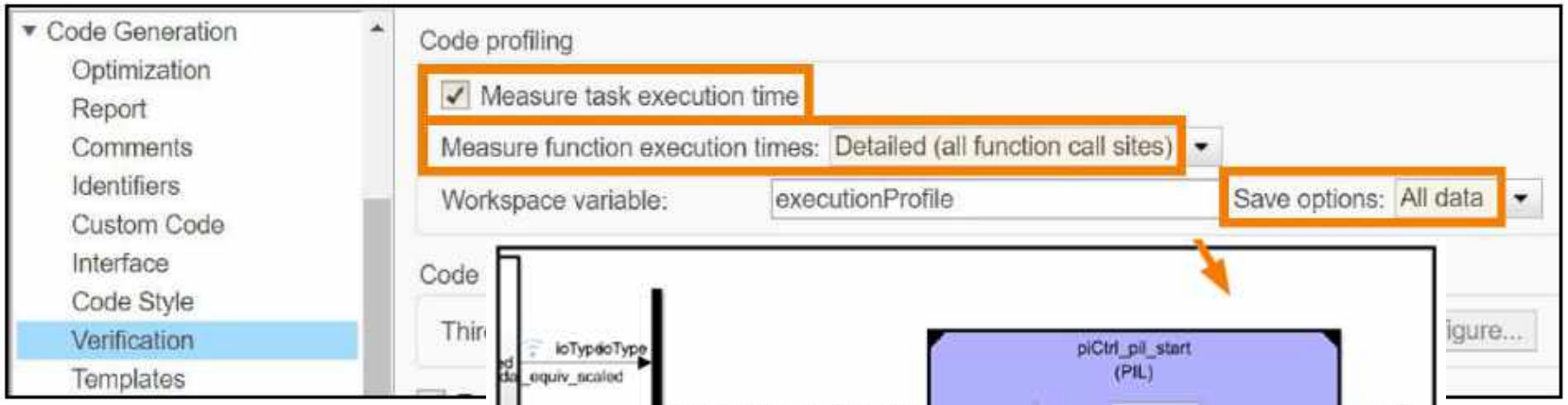
Maintainability

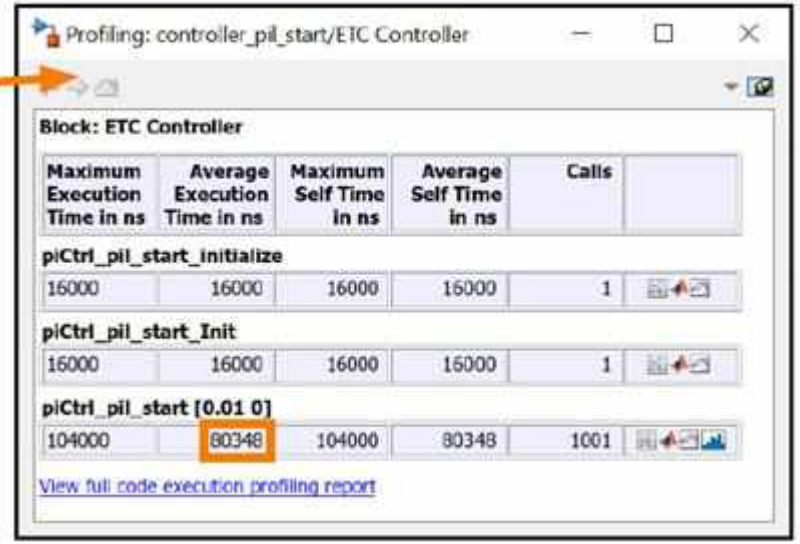
Modularity
Reusability
Analyzability
Modifiability
Testability

Model-Based Design and Automotive SPICE

SWE.3.BP4: 评估软件详细设计 - PIL

- 通过PIL (Processor-In-the-Loop)，确认目标处理器上的资源负载、性能等



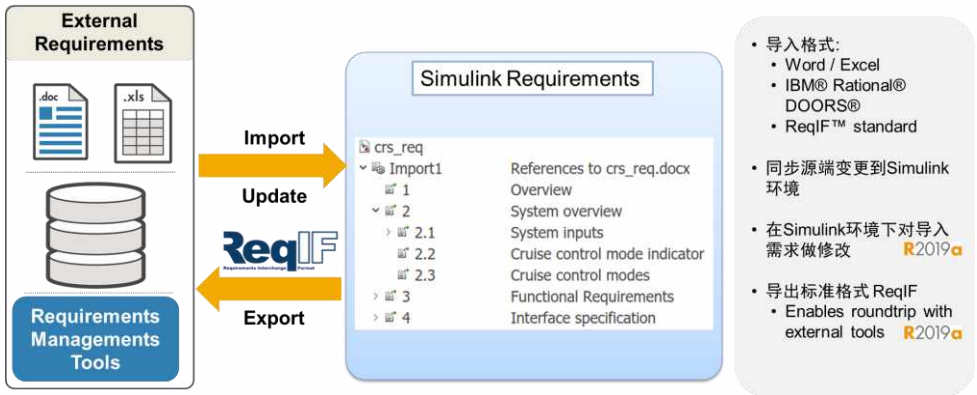


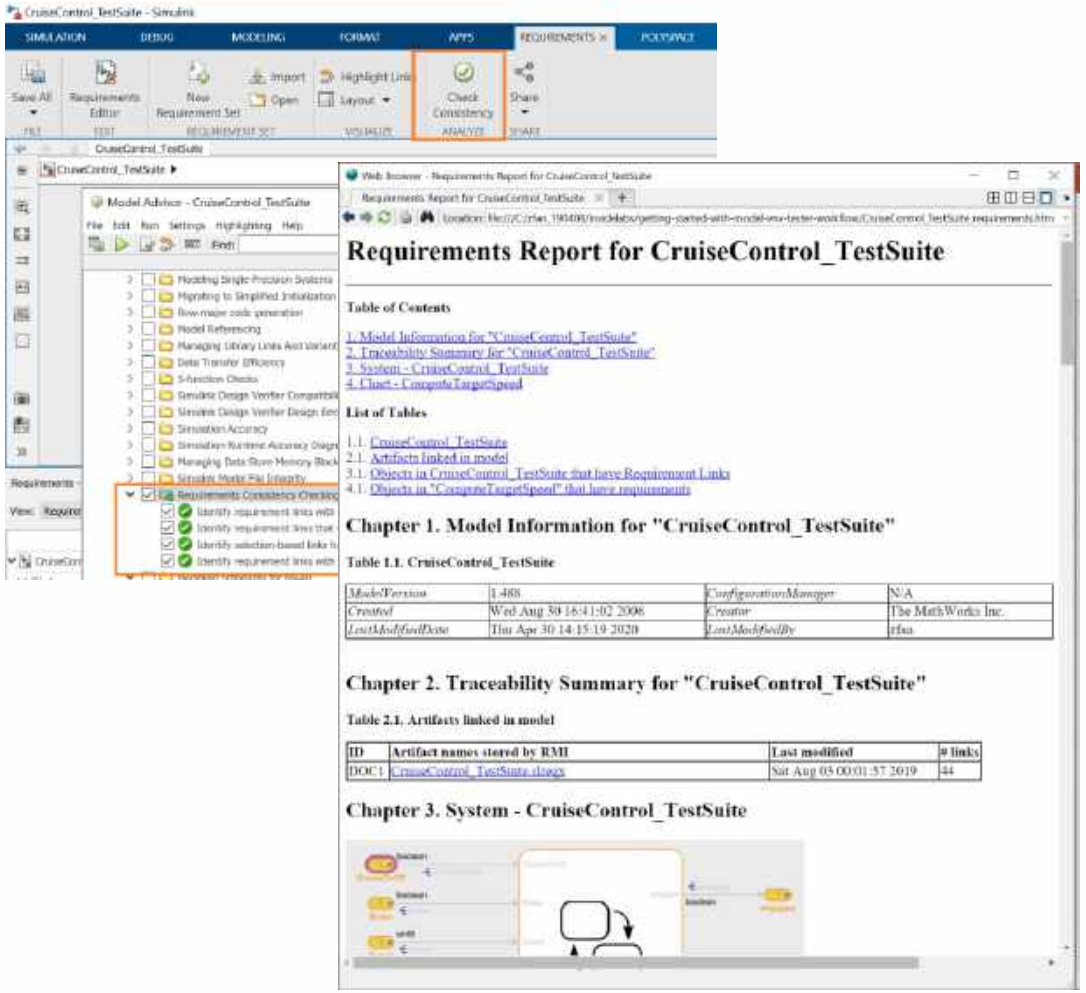
Performance efficiency
Time behavior
Resource utilization
Capacity

Model-Based Design and Automotive SPICE

SWE.3.BP5: 建立双向追溯性 & SWE.3.BP6: 确保一致性

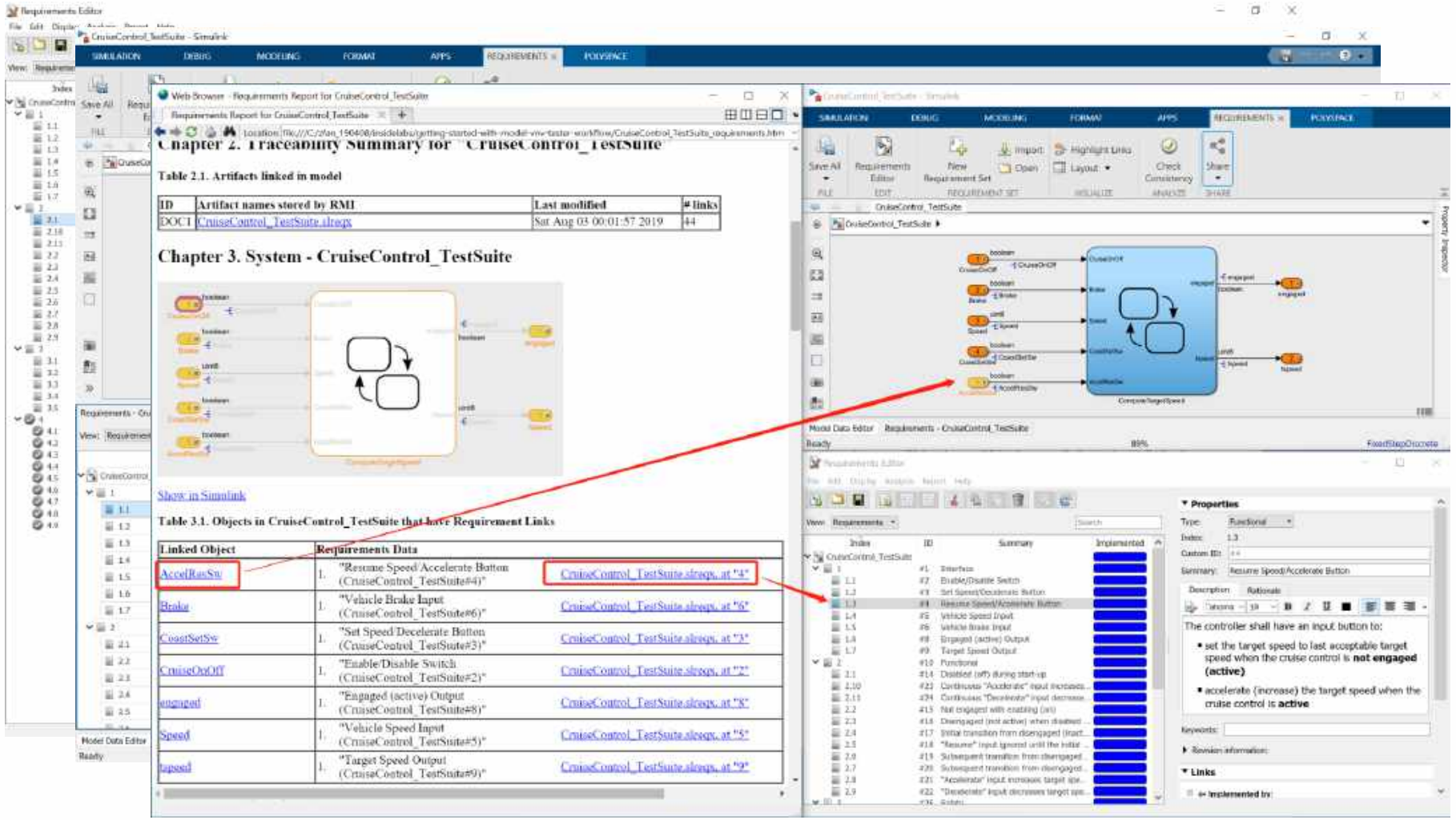
- 在模型与其设计依据之间，建立双向追溯性链接
- 设计依据的Format，可以是Word/Excel, DOORs等
- 通过traceability Report确认追溯性的完整性





Model-Based Design and Automotive SPICE

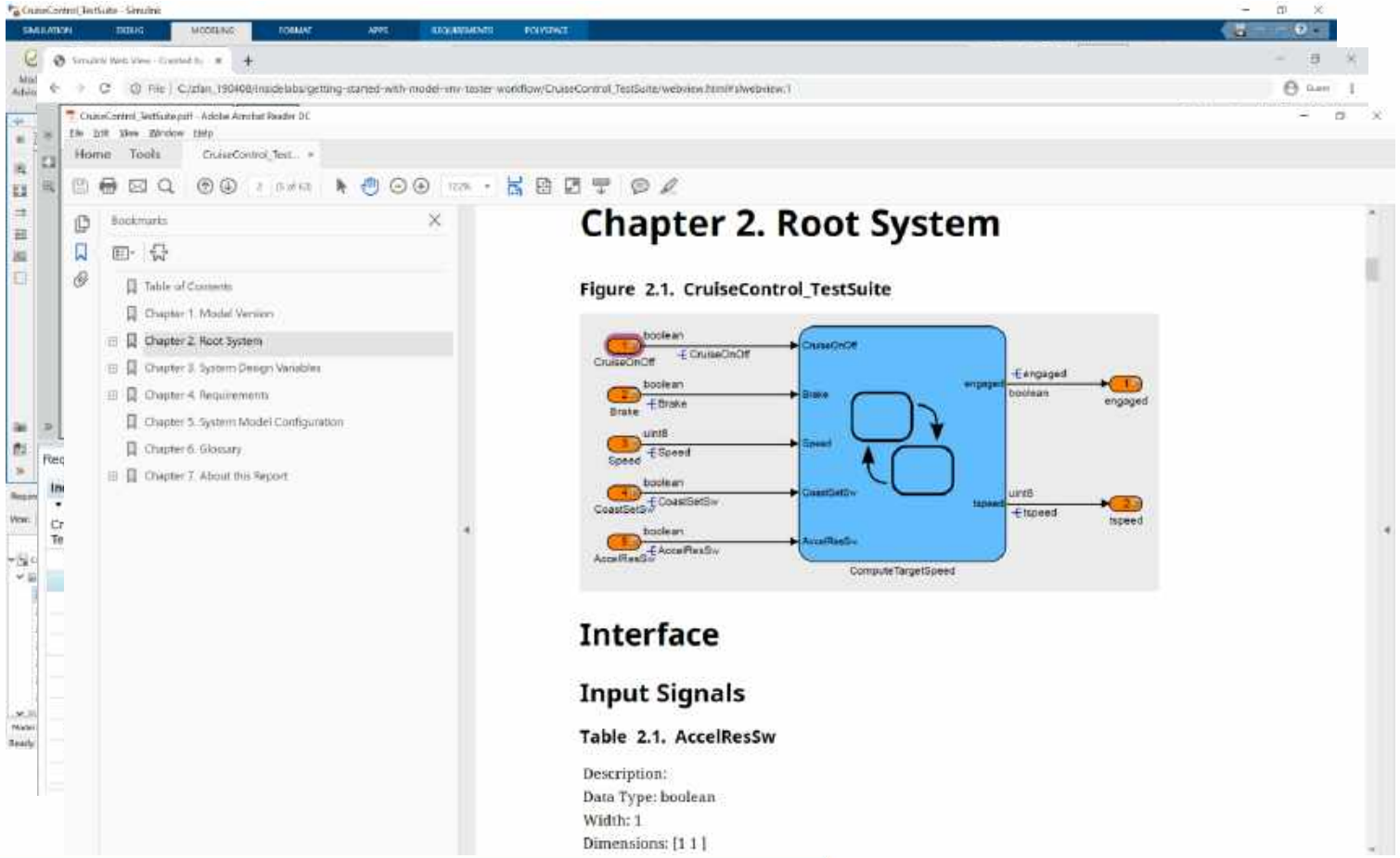
SWE.3.BP5: 建立双向追溯性 & SWE.3.BP6: 确保一致性 – 示例



Model-Based Design and Automotive SPICE

SWE.3.BP7 沟通达成一致的软件详细设计

- 通过多种形式，非常容易的在相关方之间沟通模型设计并达成一致
  - 模型
  - Web View (HTML)
  - Design Report (PDF)

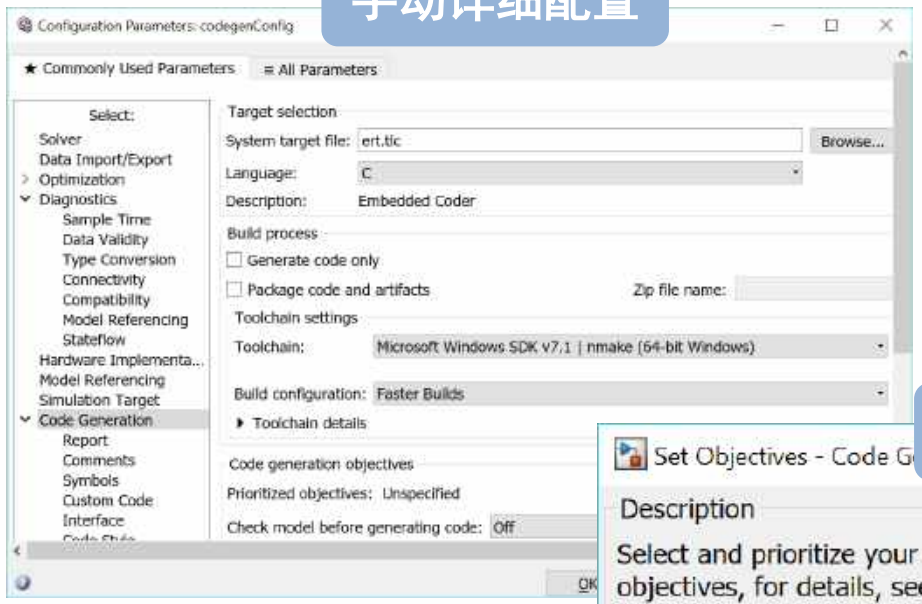


Model-Based Design and Automotive SPICE

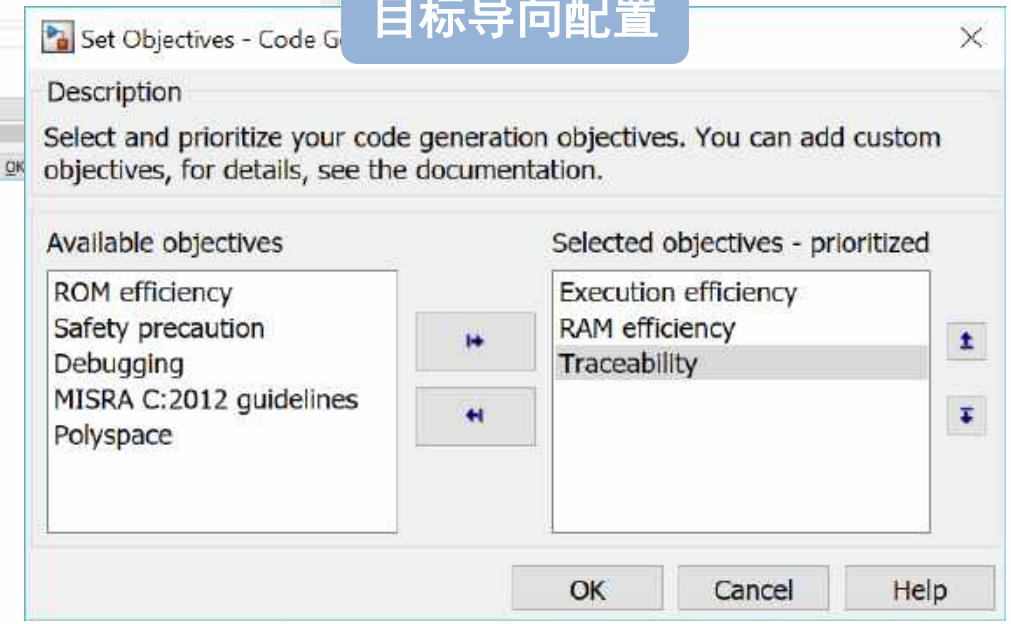
SWE.3.BP8: 构建软件单元

- MBD自动代码生成
  - 数据字典做数据管理
  - 代码生成配置
  - 优化选项

手动详细配置

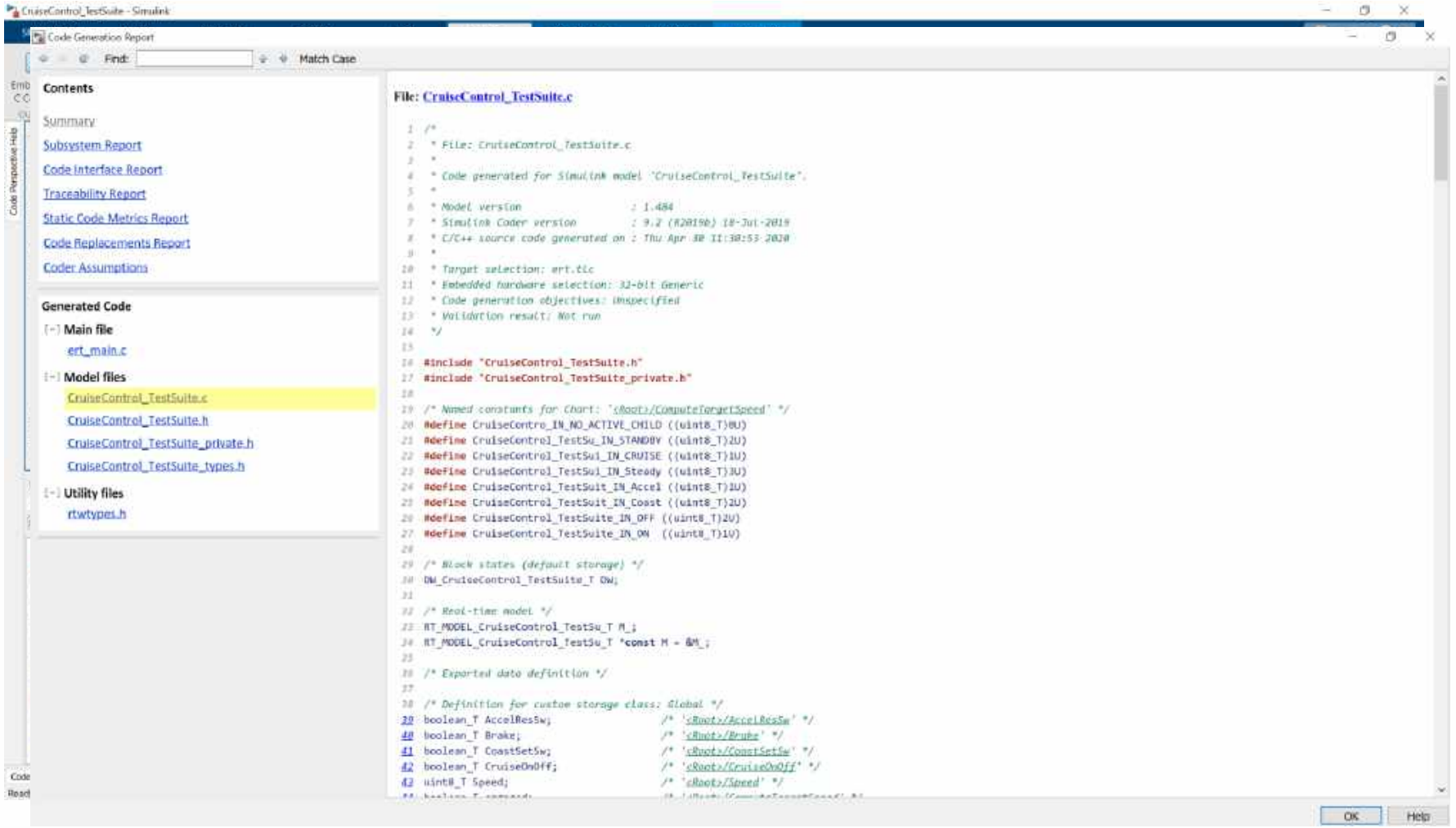


目标导向配置



Model-Based Design and Automotive SPICE

SWE.3.BP8: 构建软件单元



主题

- 背景介绍
- 采用MBD方法，满足ASPICE要求
  - MBD开发概述
  - 详细举例：SWE.3 软件详细设计与单元实现
  - 详细举例：SWE.4 软件单元测试

SWE.4 基本实践与输出

SWE.4 – 基本实践(BP)

- SWE.4.BP1: 开发包括回归策略的软件单元验证策略
- SWE.4.BP2: 开发单元验证准则
- SWE.4.BP3: 实施软件单元的静态验证
- SWE.4.BP4: 测试软件单元
- SWE.4.BP5: 建立双向追溯性
- SWE.4.BP6: 确保一致性
- SWE.4.BP7: 总结并沟通结果

实施SWE.4过程的结果如下

- 开发软件单元验证策略
- 开发软件单元验证准则
- 实施软件单元验证，并记录验证结果
- 建立了双向追溯性：
  - 软件单元和静态验证结果之间
  - 软件详细设计和单元测试规范之间
  - 软件单元测试规范和单元测试结果之间之间
- 确保一致性:
  - 软件详细设计和单元测试规范之间
- 总结验证结果，并与所有相关方沟通

Model-Based Design and Automotive SPICE

MBD场景下的测试验证概述

- 在使用模型进行设计，并基于模型自动生成代码，则[SWE.4 软件单元验证]所要求的软件单元层面的验证（静态验证+单元测试）可在模型层面上来实施。

[MBD.RL.8] If software units that are generated from the verified model by using a qualified tool chain (and without any further modification after generation) are not statically verified, the indicator SWE.4.BP3 must not be downrated.
[MBD.RL.9] If software units that are generated from the verified model by using a qualified tool chain (and without any further modification after generation) are not unit tested, the indicator SWE.4.BP4 must not be downrated.
Note: Qualified tool chain for the code generation means that there is evidence that the generated code is correct and consistent with the model.

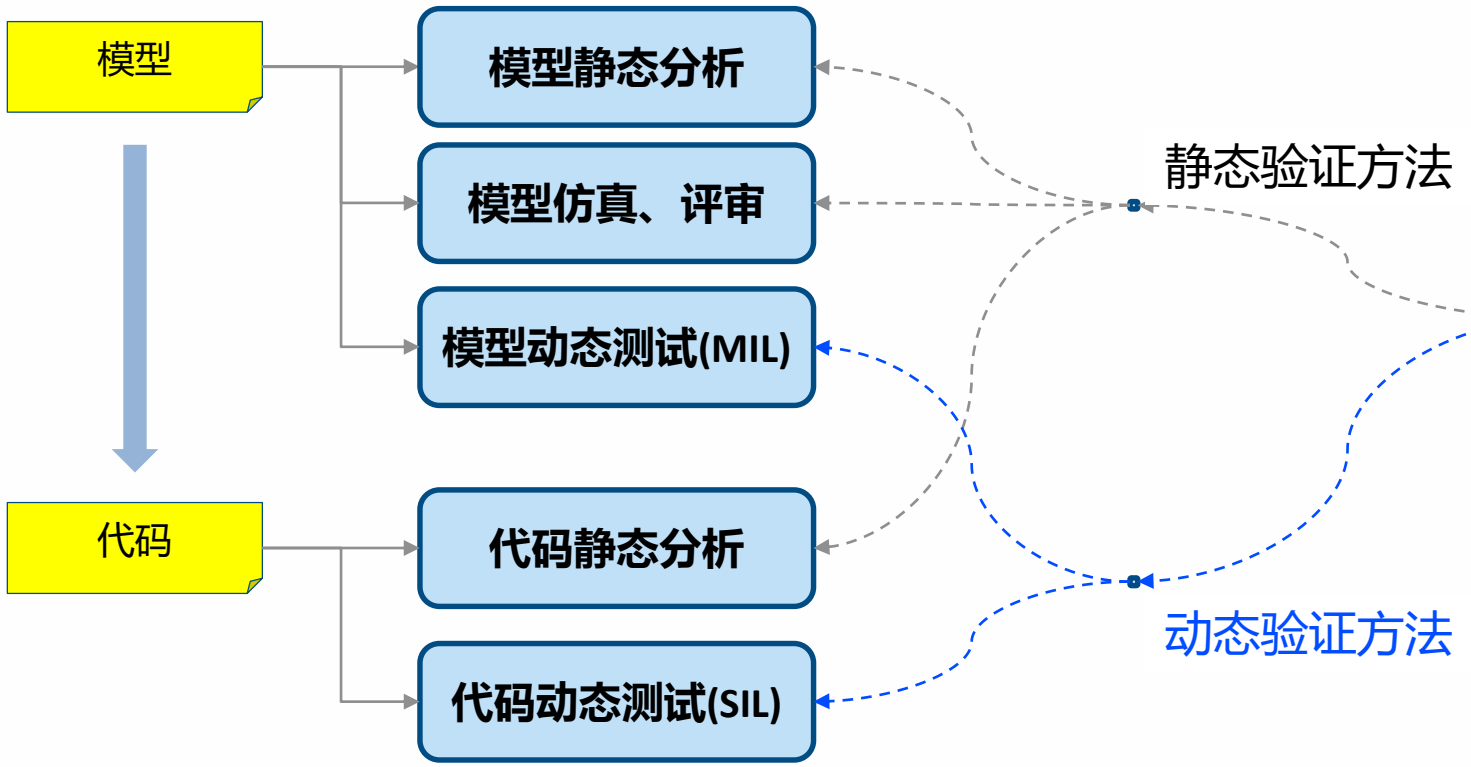
From Automotive SPICE Guideline 1st edition

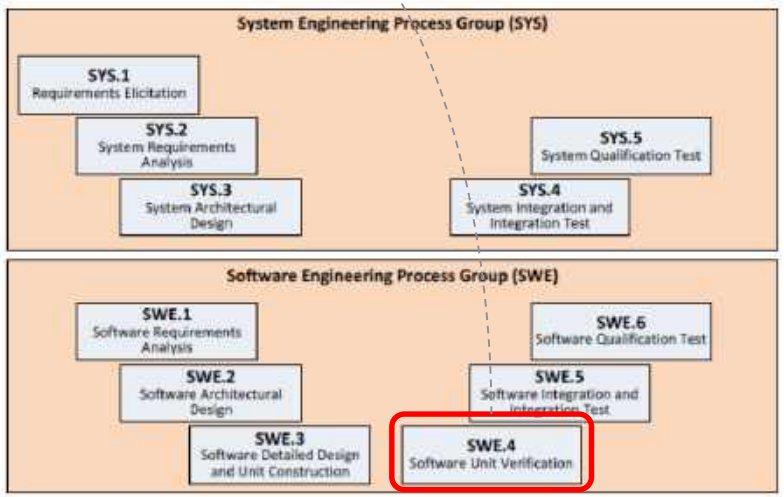
Part 6 – 9.4.1
NOTE 3 For model-based software development, the corresponding parts of the implementation model also represent objects for the verification planning. Depending on the selected software development process the verification objects can be the code derived from this model, the model itself, or both.

From ISO26262:2018

Model-Based Design and Automotive SPICE

SWE.4.BP1: 制定软件单元验证策略





Model-Based Design and Automotive SPICE

SWE.4.BP2: 开发单元验证准则 & SWE.4.BP3: 实施软件单元的静态验证

- 模型静态分析/模型仿真、评审
  - 参见SWE.3.BP4

- 代码静态分析(Polyspace)
  - 相关行业标准的符合性（如：ISO26262, MISRA等）
  - 代码质量相关指标（如：圈复杂度）
  - 形式化方法的语义分析和抽象解释，验证软件进程间、控制流和数据流行为
  - 运行时错误检查（如：溢出、被零除、数组访问越界）

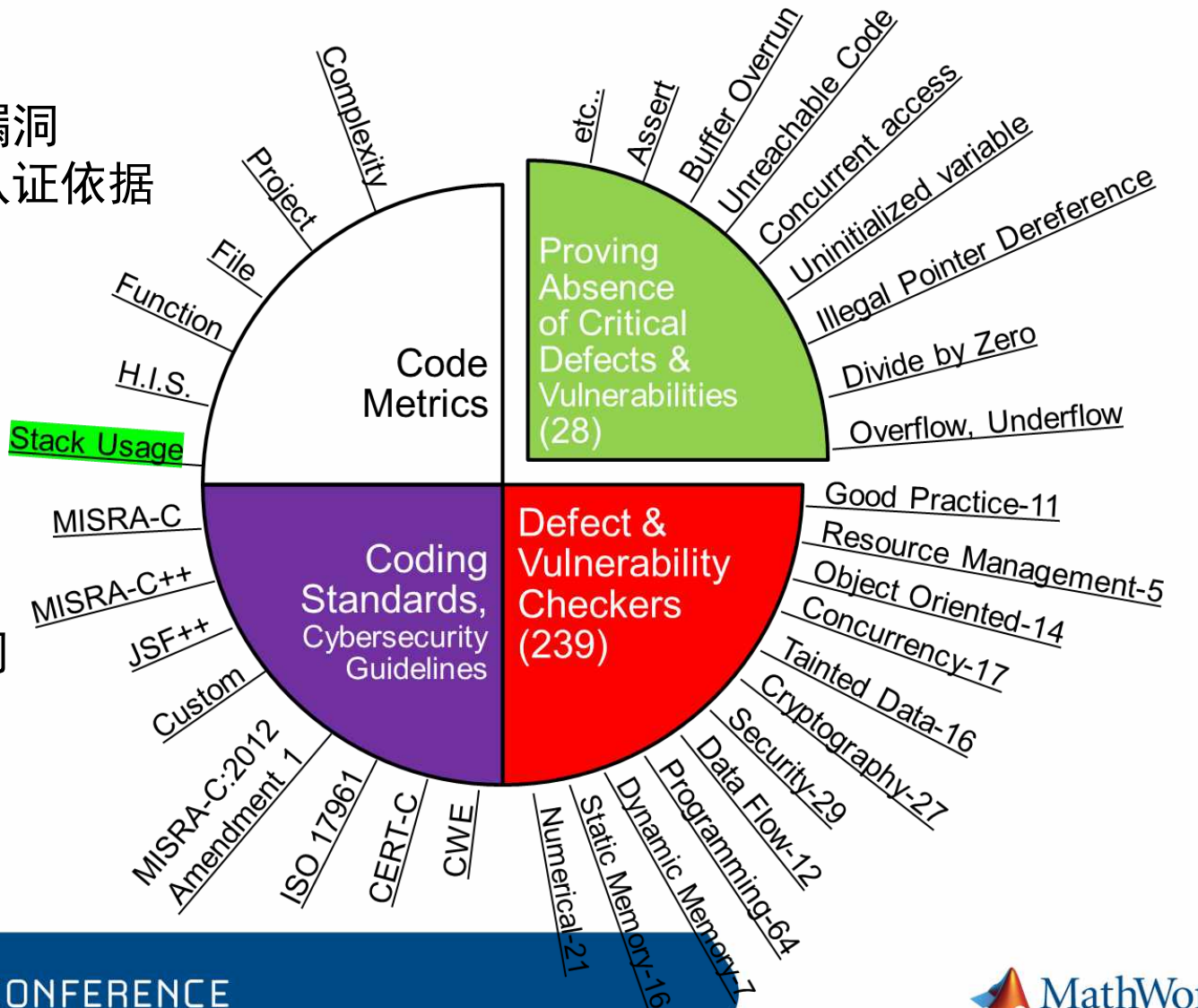
静态测试支持 - Polyspace功能一览

Bug Finder

- 保证可测量性和可维护性
- 排除绝大多数软件缺陷和漏洞
- 提供功能安全和网络安全认证依据

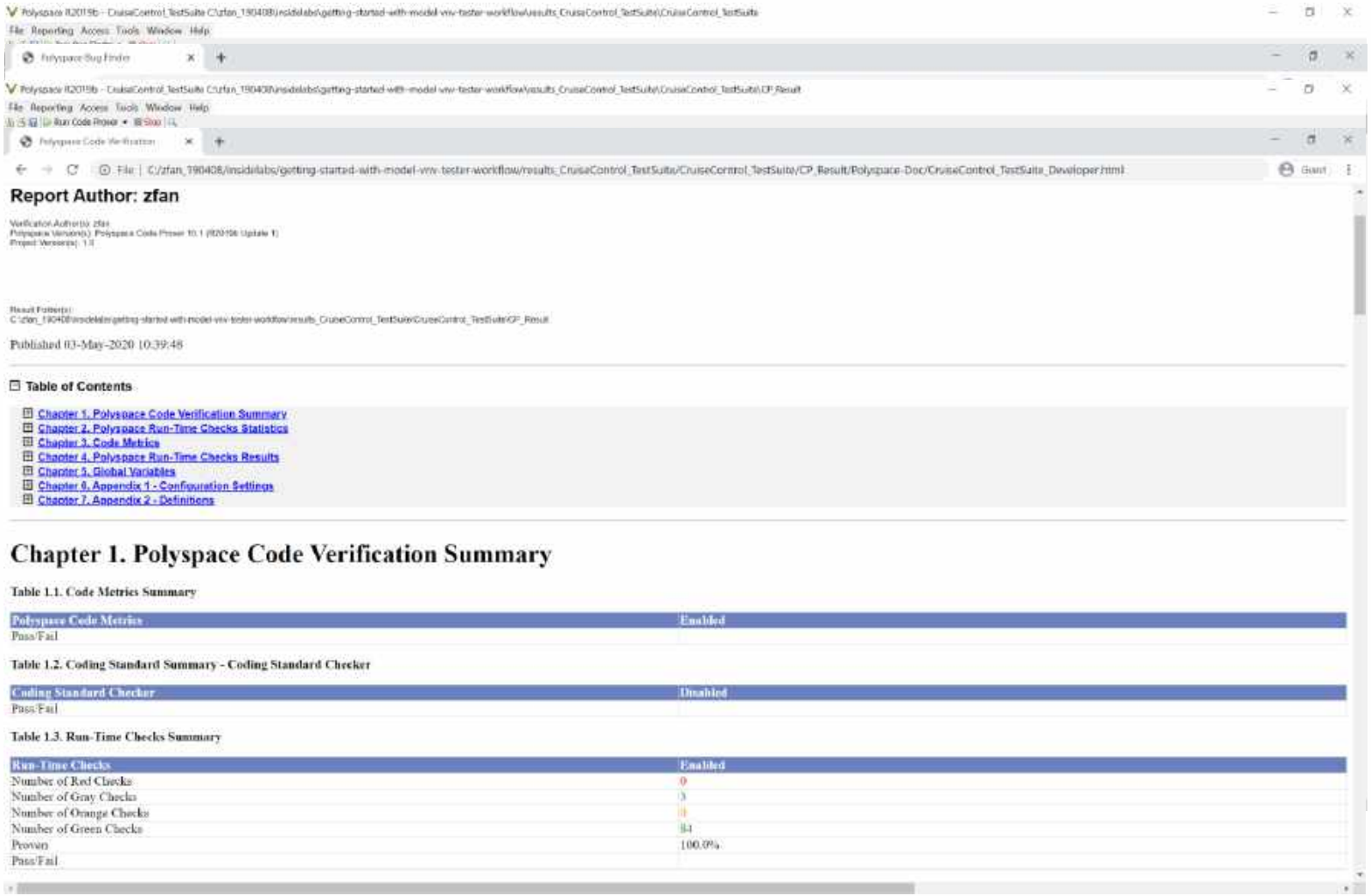
Code Prover

- 确保可靠性和安全性
- 证明无关键运行错误和漏洞
- 提供附加认证审查证据



Model-Based Design and Automotive SPICE

SWE.4.BP2开发单元验证准则 & SWE.4.BP3实施软件单元的静态验证 – 示例



Report Author: zfan

Published 03-May-2020 10:39:48

Table of Contents

- Chapter 1. Polyspace Code Verification Summary
- Chapter 2. Polyspace Run-Time Checks Statistics
- Chapter 3. Code Metrics
- Chapter 4. Polyspace Run-Time Checks Results
- Chapter 5. Global Variables
- Chapter 6. Appendix 1 - Configuration Settings
- Chapter 7. Appendix 2 - Definitions

Chapter 1. Polyspace Code Verification Summary

Table 1.1. Code Metrics Summary

Polyspace Code Metrics	Enabled
Pass/Fail	

Table 1.2. Coding Standard Summary - Coding Standard Checker

Coding Standard Checker	Disabled
Pass/Fail	

Table 1.3. Run-Time Checks Summary

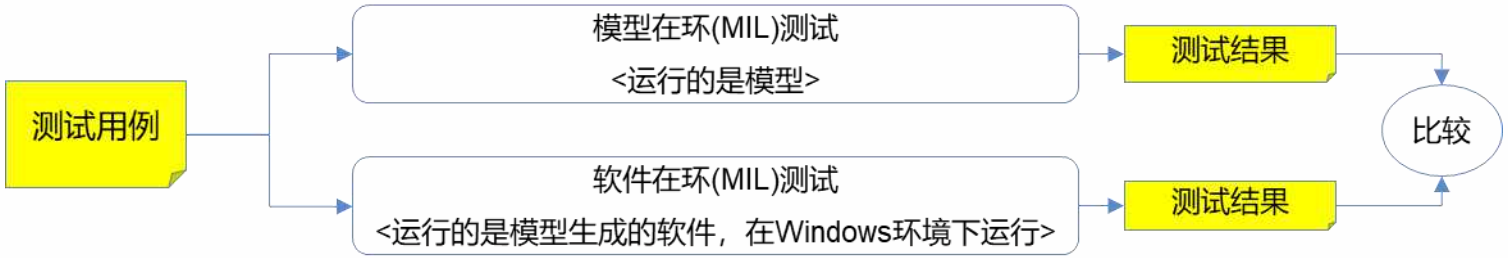
Run-Time Checks	Enabled
Number of Red Checks	0
Number of Gray Checks	3
Number of Orange Checks	0
Number of Green Checks	84
Proven	100.0%
Pass/Fail	

Model-Based Design and Automotive SPICE

SWE.4.BP2: 开发单元验证准则 & SWE.4.BP4: 测试软件单元(动态测试)

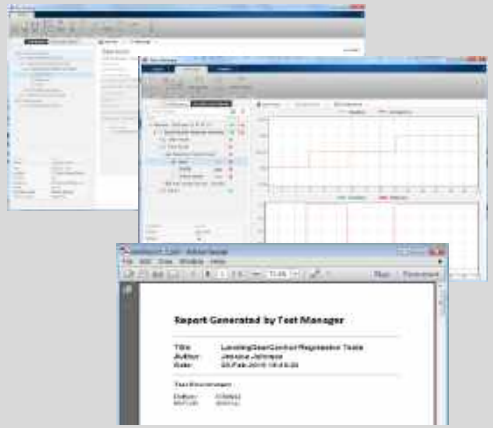
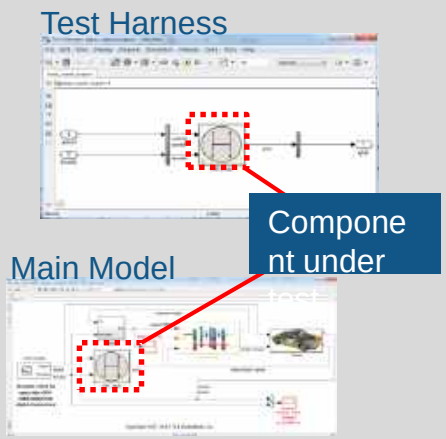
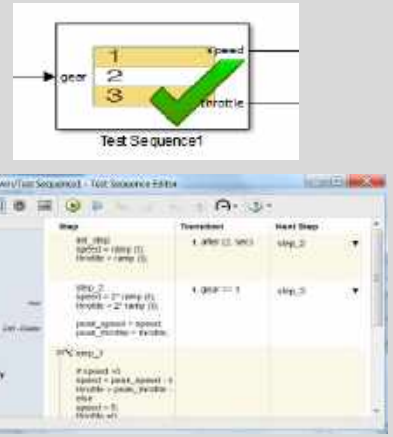
- 模型在环(MIL)测试
  - 验证SWC Req. (设计依据)是否被正确实现
  - 验证模型内部逻辑的正确性，可用结构化覆盖度指标衡量，比如MC/DC等

- 软件在环(SIL)测试
  - 验证代码与模型的等效性



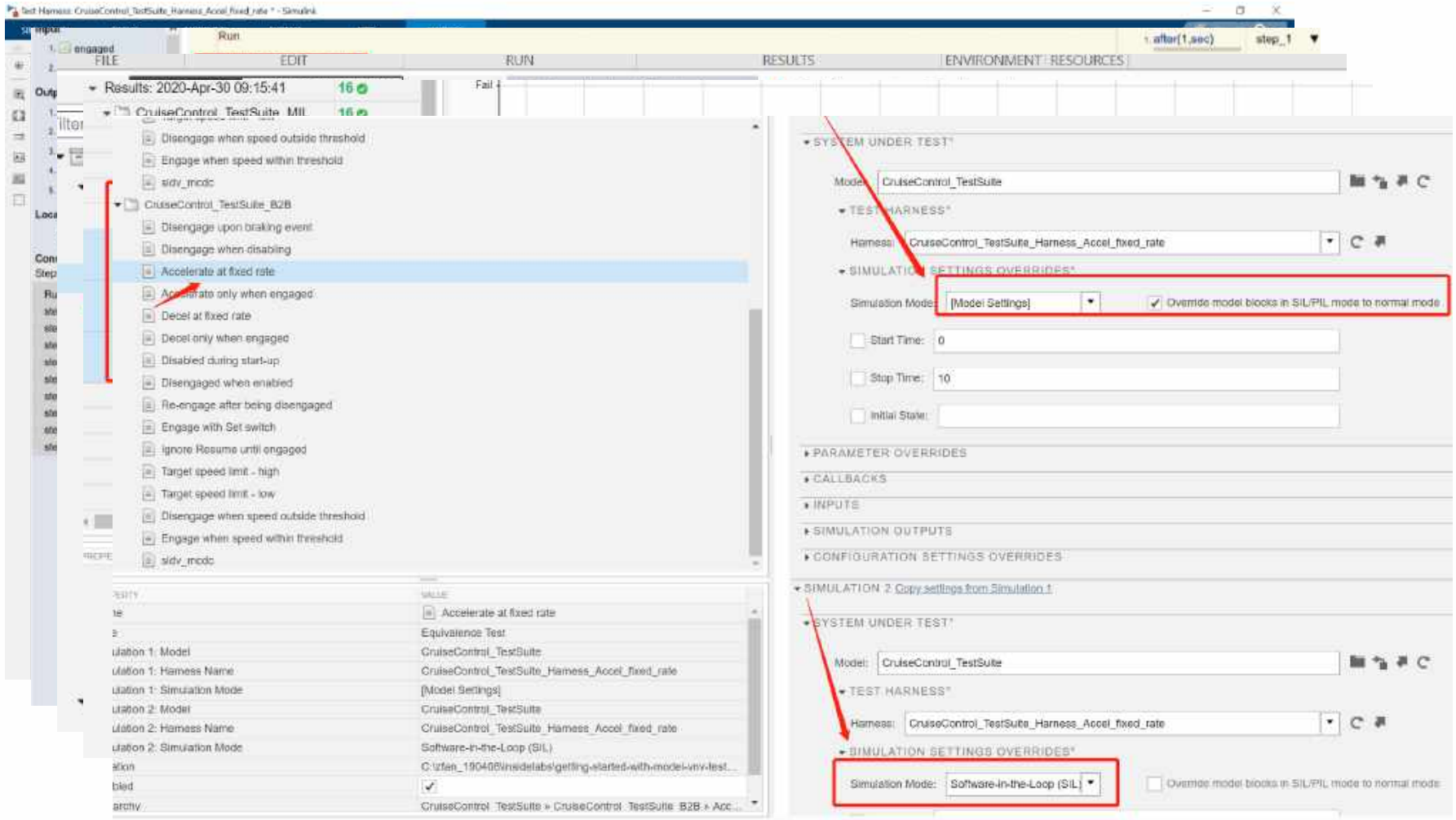
- Simulink Test
  - Test Manager, Test Sequence, Test Harness等协助进行测试管理、测试用例设计、测试执行等

动态测试支持 - Simulink Test

Test Manager	Test Harness	Test Sequence Block
• Author, execute, manage test cases • Review, export, report	• Synchronized, simulation test environment	• Test Inputs and assessments • Based on logical, temporal conditions
		

Model-Based Design and Automotive SPICE

SWE.4.BP2: 开发单元验证准则 & SWE.4.BP4: 测试软件单元(动态) – 示例



Model-Based Design and Automotive SPICE

SWE.4.BP5: 建立双向追溯性 & SWE.4.BP6: 确保一致性

- 工具能建立：
  - 测试用例与SWC Req.之间的追溯性
  - 测试用例与测试结果之间的追溯性
  - 静态分析结果与代码之间的关联

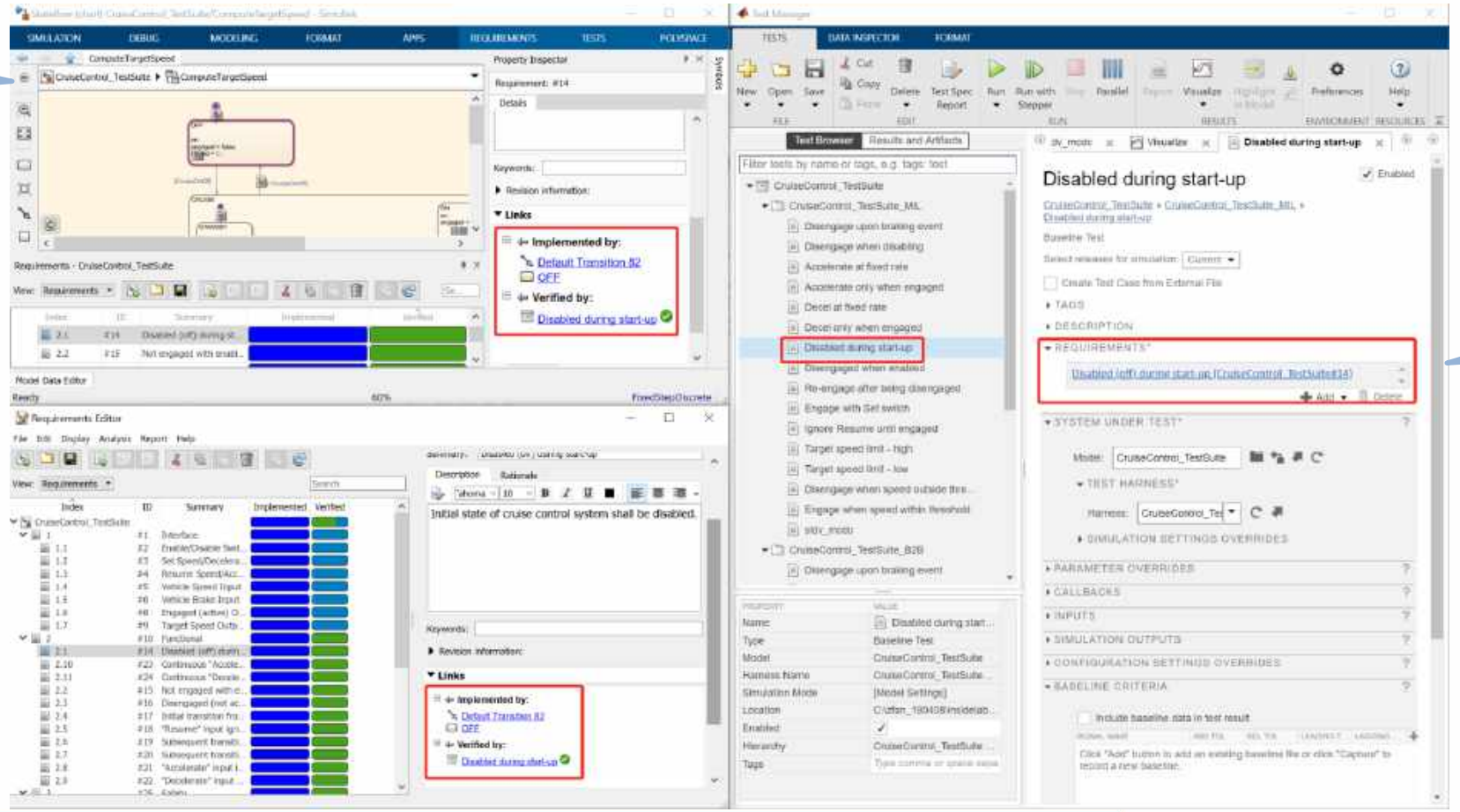
- 基于如上的追溯性链接，方便确认追溯项之间的一致性

Model-Based Design and Automotive SPICE

SWE.4.BP5: 建立双向追溯性 & SWE.4.BP6: 确保一致性 (动态测试)

需求透视图

需求编辑视图

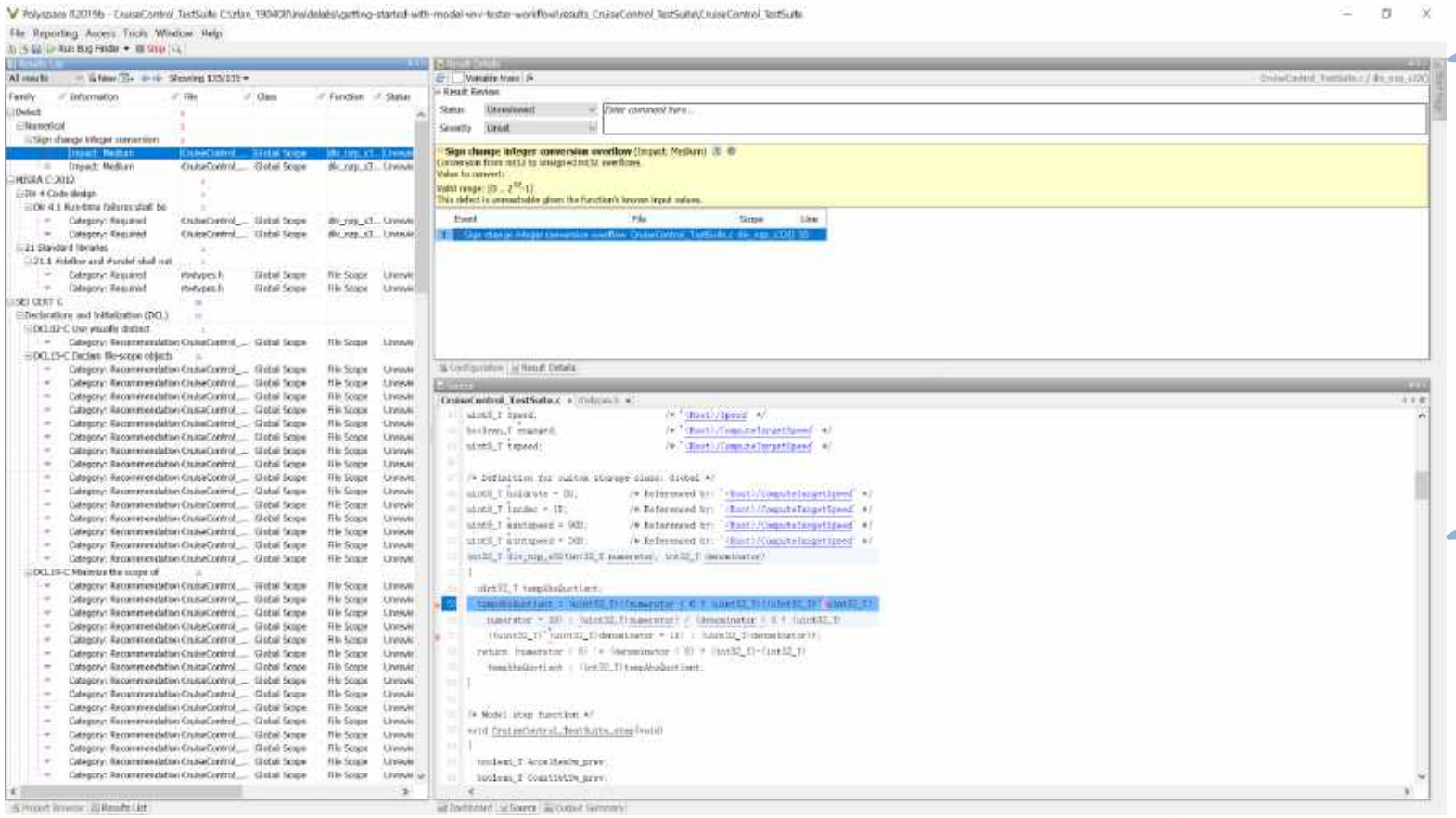


测试用例视图

Model-Based Design and Automotive SPICE

SWE.4.BP5: 建立双向追溯性 & SWE.4.BP6: 确保一致性 (静态分析)

Polyspace 结果列表



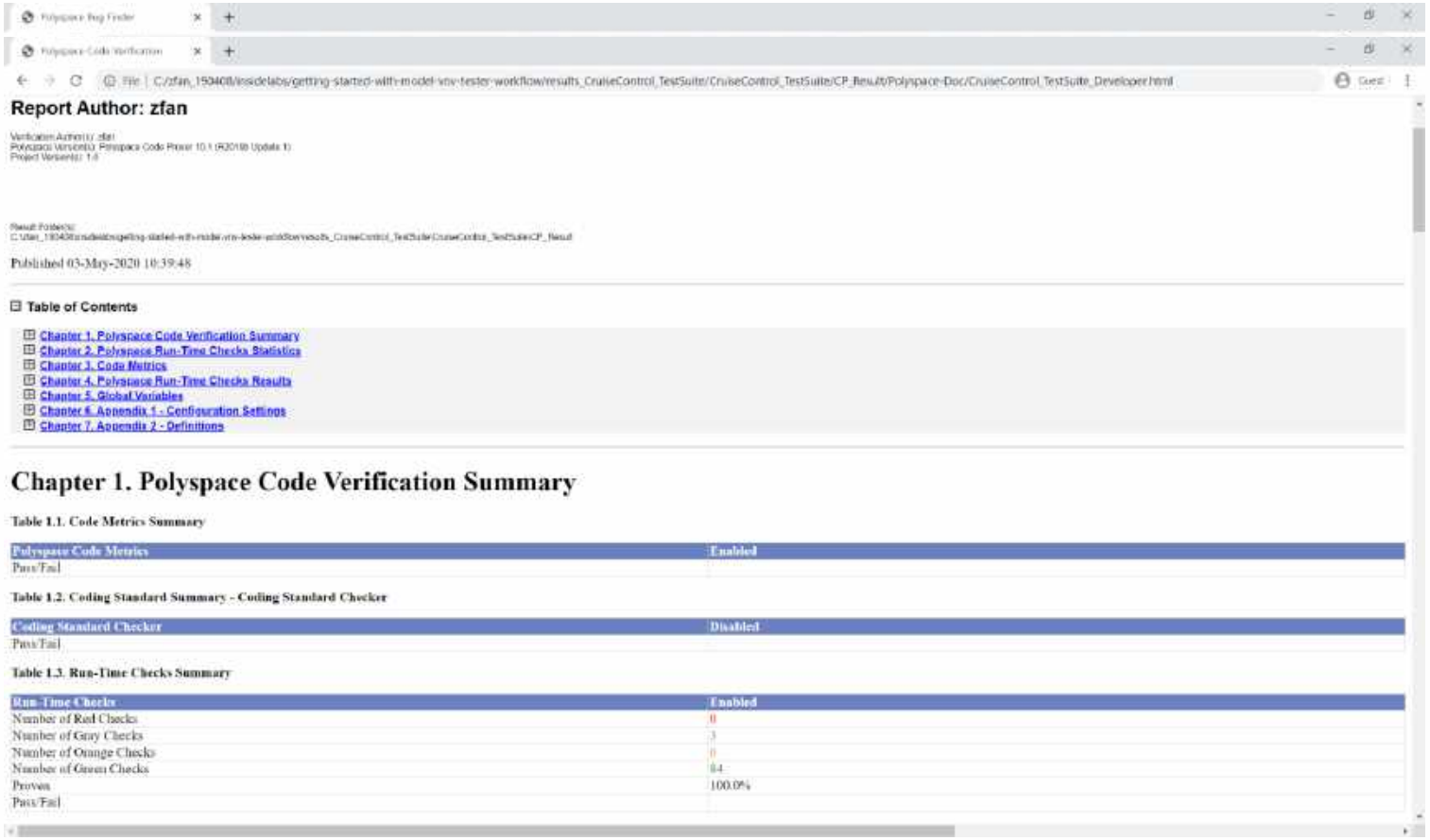
Polyspace 结果说明

Polyspace 代码区域

Model-Based Design and Automotive SPICE

SWE.4.BP7: 总结并沟通结果(静态分析)

- 多种形式的静态分析和动态测试的验证结果，方便在相关方中沟通单元验证结果。



Report Author: zfan

Published 03-May-2020 10:39:48

Table of Contents

- Chapter 1. Polyspace Code Verification Summary
- Chapter 2. Polyspace Run-Time Checks Statistics
- Chapter 3. Code Metrics
- Chapter 4. Polyspace Run-Time Checks Results
- Chapter 5. Global Variables
- Chapter 6. Appendix 1 - Configuration Settings
- Chapter 7. Appendix 2 - Definitions

Chapter 1. Polyspace Code Verification Summary

Table 1.1. Code Metrics Summary

Polyspace Code Metrics	Enabled
Pass/Fail	

Table 1.2. Coding Standard Summary - Coding Standard Checker

Coding Standard Checker	Disabled
Pass/Fail	

Table 1.3. Run-Time Checks Summary

Run-Time Checks	Enabled
Number of Red Checks	0
Number of Gray Checks	3
Number of Orange Checks	0
Number of Green Checks	84
Proven	100.0%
Pass/Fail	

Model-Based Design and Automotive SPICE

SWE.4.BP7: 总结并沟通结果 (动态测试)

Report Generated by Test Manager

Title: Test
Author: zfan
Date: 03-May-2020 10:13:44

Test Environment

Platform: PCWIN64
MATLAB: (R2019b)

Summary

Name	Outcome	Duration (Seconds)
Results: 2020-May-03 09:12:30	16	213.282
CruiseControl_TestSuite_MIL	16	213.282
Disengage upon braking event		14.04
Disengage when disabling		15.485
Accelerate at fixed rate		8.314
Accelerate only when engaged		12.187
Decel at fixed rate		14.357
Decel only when engaged		12.788
Disabled during start-up		14.404
Disengaged when enabled		12.232
Re-engage after being disengaged		15.254
Engage with Set switch		13.204
Ignore Resume until engaged		13.375
Target speed limit - high		12.55
Target speed limit - low		12.751
Disengage when speed outside threshold		13.841
Engage when speed within threshold		12.987
sldv_mcdc		15.228
Test Case 1		15.23

Disengage upon braking event

Test Result Information

Result Type: Test Case Result
Parent: CruiseControl_TestSuite_MIL
Start Time: 2020-05-03 09:12:35
End Time: 2020-05-03 09:12:49
Outcome: Passed

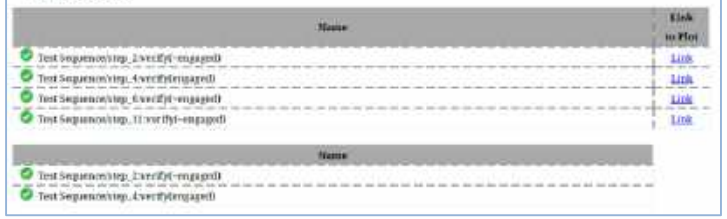
Test Case Information

Name: Disengage upon braking event
Type: Baseline Test

Test Case Requirements

Description: Vehicle braking will transition system to disengaged (inactive) when engaged (active) (CruiseControl_TestSuite#26)
Document: CruiseControl_TestSuite.slreqx
Description: Disengage when braking occurs (CruiseControl_TestSuite#34)
Document: CruiseControl_TestSuite.slreqx

Verify Result



Simulation

System Under Test Information

Model: CruiseControl_TestSuite
Harness: CruiseControl_TestSuite_Harness_Disengage_upon_braking
Harness Owner: CruiseControl_TestSuite
Simulation Mode: normal
Override SIL or PIL Mode: 0
Configuration Set: ModelReferencingVisual

Start Time: 0
Stop Time: 10
Checksum: 1485891064 2364595699 664815567 2209392514
Simulink Version: 10.0
Model Version: 1.14
Model Author: patcanny
Date: Sun May 03 09:11:47 2020
User ID: zfan
Model Path: C:\zfan_190408\Insidelabs\getting-started-with-model-vnv-tester-workflow\Tests\Harnesses\CruiseControl_TestSuite_Harness_Disengage_upon_braking.slx
Machine Name: SHA-ZFAN
Solver Name: FixedStepDiscrete
Solver Type: Fixed-Step
Fixed Step Size: 0.100000000000000001
Simulation Start Time: 2020-05-03 09:12:38
Simulation Stop Time: 2020-05-03 09:12:44
Platform: PCWIN64

MathWorks A-SPICE 解决方案概述

Overall mapping A-SPICE to MathWorks solution

Process Group		MathWorks Solution										
		Simulink	StateFlow	Embedded, Simulink Coder	Simulink Requirements	System Composer	Simulink Test	Simulink Check	Simulink Design Verifier	Simulink Coverage	Polyspace Bug Finder	Polyspace Code Prover
System Engineering Process Group	System Requirements Analysis				○							
	System Architectural Design				○	○						
	System Integration/ Integration Test				○		○					
	System Qualification Test				○		○					
Software Engineering Process Group	Software Requirements Analysis				○							
	Software Architectural Design				○	○						
	Software Detailed Design	○	○				○	○	○			
	Unit Construction			○								
	Software Unit Verification						○			○	○	○
	Software Integration and Integration Test				○		○					
	Software Qualification Test				○		○					